



NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST SCHOOL OF COMPUTING — KARACHI CAMPUS

VIBE SHIFT

AI-Powered Music Transformation

Final Year Project Report

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Artificial Intelligence

PROJECT SUPERVISOR Mr. Zain-ul-Hasan

PROJECT CO-SUPERVISOR Mrs. Sania Urooj

PROJECT TEAM Mehreen Saghar 22k-4071
Insiya Fakhruddin 22k-4103
Syeda Fatima Zafar 22k-4124

SUBMISSION DATE May 10, 2026

DEPARTMENT OF COMPUTER SCIENCE
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
Spring 2026

Declaration of Authorship

We, Mehreen Saghar, Insiya Fakhruddin, and Syeda Fatima Zafar, declare that this report titled *VibeShift — AI-Powered Music Transformation* and the work presented in it are our own. We confirm that:

- This work was done wholly or mainly while in candidature for a degree at this University.
- Where we have consulted the published work of others, this is always clearly attributed.
- Where we have quoted from the work of others, the source is always given. With the exception of such quotations, this report is entirely our own work.
- We have acknowledged all main sources of help.
- Where the thesis is based on work done by us jointly with others, we have made clear exactly what was done by others and what we have contributed ourselves.

Mehreen Saghar

22k-4071

Insiya Fakhruddin

22k-4103

Syeda Fatima Zafar

22k-4124

Document Information

Category	Information
Customer	FAST — NUCES Karachi Campus
Project Title	VibeShift — AI-Powered Music Transformation
Document	Final Year Project Report
Status	Completed
Authors	Mehreen Saghar (22k-4071), Insiya Fakhruddin (22k-4103), Syeda Fatima Zafar (22k-4124)
Project Supervisor	Mr. Zain-ul-Hasan
Co-Supervisor	Mrs. Sania Urooj
Issue Date	10th May, 2026

Abstract

VibeShift is an AI-powered mobile application for music transformation, designed to democratise professional-grade audio processing for non-technical users on iOS and Android. The system implements two core modules: a stem separation module using Facebook's Demucs HTDemucs neural network, which separates any audio track into four constituent stems (vocals, drums, bass, and other instrumentation); and a genre transformation module that converts audio between musical genres through an intelligently orchestrated pipeline.

The genre transformation module was developed in two phases. In the research phase, a CycleGAN model was trained from scratch on the GTZAN music dataset (1,000 songs, 10 genres) using mel-spectrogram representations, demonstrating the feasibility of spectrogram-domain genre transfer while revealing fundamental limitations in audio quality and scalability. In the production phase, Facebook's MusicGen-Melody model, deployed via the Replicate cloud API, was adopted as the final solution, achieving significantly superior audio quality and semantic genre accuracy across all 10 genres.

The system is built on a React Native / Expo 54 mobile frontend, a FastAPI 0.104 Python backend, Clerk RS256 JWT authentication, AWS S3 audio storage, and a five-table SQLite/PostgreSQL database. Evaluation results demonstrate a Reconstruction SDR of +29.84 dB for stem separation, confirming near-lossless source separation, and an average subjective quality score of 8.05/10 for MusicGen-based genre transformation versus 5.5/10 for the CycleGAN approach.

Table of Contents

Declaration of Authorship

Document Information

Abstract

1.0 Introduction

- 1.1 Purpose of Document
- 1.2 Intended Audience
- 1.3 Scope
- 1.4 Problem Statement
- 1.5 Project Objectives
- 1.6 Gap Analysis
- 1.7 Document Organisation

2.0 Related Work

- 2.1 Chapter Overview
- 2.2 Audio Source Separation Techniques
- 2.3 AI-Based Genre Transfer and Style Transfer
 - 2.3.1 GAN-Based Spectrogram Translation
 - 2.3.2 Generative Audio Models
- 2.4 Existing Music Editing Platforms
- 2.5 Comparative Analysis
- 2.6 Positioning VibeShift

3.0 Requirements

- 3.1 Chapter Overview
- 3.2 Functional Requirements
- 3.3 Non-Functional Requirements
- 3.4 Use Cases
- 3.5 System Constraints and Assumptions

4.0 Design

- 4.1 Chapter Overview
- 4.2 System Architecture
- 4.3 Database Design
- 4.4 Application Design
- 4.5 UI/UX Design
- 4.6 Key Code Snippets

5.0 Implementation

- 5.1 Chapter Overview
- 5.2 Technology Stack
- 5.3 Frontend Implementation
- 5.4 Backend Implementation
- 5.5 AI Microservices Implementation
- 5.6 Database Implementation
- 5.7 Replicate API Integration

6.0 Testing and Evaluation

- 6.1 Chapter Overview

- 6.2 Demixer Testing and Evaluation
- 6.3 CycleGAN Testing and Evaluation
- 6.4 MusicGen Testing and Evaluation
- 6.5 Comparative Analysis
- 6.6 Final Conclusion

7.0 Conclusion

- 7.1 Summary of Achievements
- 7.2 Challenges and Learnings
- 7.3 Future Work

References

Generative AI Usage Statement

1.0 Introduction

1.1 Purpose of Document

This document constitutes the Final Year Project (FYP) report for VibeShift, an AI-powered mobile application for music transformation. It provides a comprehensive technical and academic account of the system's conception, design, implementation, evaluation, and deployment, intended to satisfy the requirements of the Final Year Project module and to serve as a complete reference for all project stakeholders.

The report covers: the motivation and problem context behind the project; a review of related academic literature and existing commercial solutions; a detailed specification of functional and non-functional requirements; the architectural and database design of the platform; a full account of each implementation component; the testing strategy and quantitative AI model evaluation; and a critical reflection on achievements, challenges, and future directions. IEEE citation style is followed throughout.

1.2 Intended Audience

This report is written for academic supervisors and examiners assessing technical depth and engineering quality; software engineers and AI practitioners who may extend the system; music technology researchers interested in deep learning applied to audio; mobile developers referencing the React Native implementation; and future students undertaking similar projects. Readers are assumed to have working knowledge of software engineering principles and basic machine learning concepts.

1.3 Scope

VibeShift is a cross-platform mobile application for iOS and Android providing two core AI-powered services: stem separation and genre transformation.

In scope:

- React Native / Expo mobile frontend for audio upload, stem visualisation, editing, genre selection, and download.
- Python FastAPI backend with Clerk authentication, AWS S3 file storage, and ML service orchestration.
- Stem separation using Facebook's Demucs model producing four stems: vocals, drums, bass, and other.
- Per-stem pitch shifting (± 12 semitones) and timbre modification via HPSS decomposition.
- Genre transformation via two approaches: CycleGAN trained on GTZAN (research phase) and MusicGen via Replicate API (production phase).
- Vocal preservation pipeline with genre-specific FX, BPM-based tempo matching, and parametric re-mixing.

- Five-table relational database managing users, demix jobs, stems, mix exports, and transform jobs.
- Quantitative evaluation of both modules using signal processing metrics.

Out of scope: real-time audio processing, in-app recording, a web frontend, MIDI output, and copyright management.

1.4 Problem Statement

Sophisticated audio manipulation — separating instruments from a mixed recording, or transforming a song's genre — has historically required expensive professional software, specialised hardware, and years of expertise, placing these capabilities beyond the reach of most music enthusiasts. While deep learning has produced powerful models such as Demucs [1] for source separation and MusicGen [2] for generative music synthesis, a critical gap remains between research-grade capabilities and accessible mobile applications that non-technical users can operate on everyday devices.

Five specific problems motivate VibeShift:

- **Accessibility Barrier:** Professional DAWs such as Pro Tools and Ableton Live cost \$200–\$750 and require significant expertise. Mobile alternatives lack AI-driven stem separation and genre transformation.
- **Fragmentation of Tools:** Users must navigate multiple disconnected platforms — stem separators, audio editors, and music generators — with no integrated workflow between them, introducing friction and quality loss at each transfer.
- **Technical Inaccessibility of AI Models:** State-of-the-art models including Demucs, MusicGen, and CycleGAN exist as research codebases requiring GPU infrastructure and command-line expertise, making them inaccessible to general users.
- **Genre Transformation Gap:** No production-ready mobile application allows a user to upload a song, select a target genre, and receive a convincingly transformed version on their smartphone.
- **Vocal Preservation in Genre Transfer:** Existing style transfer systems operate on the full audio mixture, producing vocal artefacts and loss of lyrical intelligibility. A system that explicitly separates, preserves, and re-integrates vocals during genre transformation is a significant practical contribution.

VibeShift addresses all five problems by delivering a free, intuitive mobile application that makes professional-grade AI audio capabilities available to any smartphone user without technical knowledge or expensive hardware.

1.5 Project Objectives

1.5.1 Technical Objectives

- Implement a stem separation pipeline using Demucs achieving Reconstruction SDR exceeding 10 dB across test audio.

- Implement per-stem pitch shifting (± 12 semitones via librosa) and timbre modification (HPSS decomposition) applicable independently to any stem.
- Train and evaluate a CycleGAN genre transformation model on the GTZAN dataset using mel-spectrogram representations, documenting its capabilities and limitations.
- Migrate the production genre transformation pipeline to MusicGen-Melody via Replicate API, supporting all 10 GTZAN genres through text prompt conditioning.
- Implement a multi-stage vocal preservation pipeline: Demucs separation $\rightarrow$ genre-specific FX $\rightarrow$ MusicGen instrumental generation $\rightarrow$ BPM tempo matching $\rightarrow$ parametric re-mixing.
- Build a FastAPI backend with Clerk RS256 JWT authentication, AWS S3 storage, and asynchronous background task processing.
- Design a five-table normalised database schema (users, demix_jobs, stems, demix_mixes, transform_jobs) using SQLAlchemy.
- Develop a React Native / Expo mobile application for the complete user workflow from authentication through to audio download.

1.5.2 Evaluation Objectives

- Evaluate stem separation using Reconstruction SDR, Dynamic SNR, Spectral Convergence, and Energy Distribution across three test songs.
- Conduct a structured comparative evaluation of CycleGAN versus MusicGen across training time, inference latency, audio quality, and scalability dimensions.

1.5.3 Engineering Objectives

- Deploy a Dockerised backend on AWS EC2 with CloudFormation infrastructure provisioning and S3 audio storage.
- Implement Clerk JWKS-based RS256 JWT verification with public key caching for secure, scalable mobile authentication.

1.6 Gap Analysis

Table 1.1 compares VibeShift against the most relevant existing platforms across nine capability dimensions.

Feature	LALAL.AI	Spleeter	Moises	Soundraw	GarageBand	VibeShift
Stem Separation	✓	✓	✓	✗	Partial	✓
Per-Stem Pitch Edit	✗	✗	✓	✗	✓	✓
Per-Stem Timbre Edit	✗	✗	✗	✗	✗	✓
AI Genre Transformation	✗	✗	✗	Partial	✗	✓
Vocal Preservation	✗	✗	✗	✗	✗	✓
Mobile App (iOS & Android)	✓	✗	✓	✓	iOS only	✓

Feature	LALAL.AI	Spleeter	Moises	Soundraw	GarageBand	VibeShift
Free to Use	Freemium	✓	Freemium	Freemium	✓	✓
No Technical Setup	✓	✗	✓	✓	✓	✓
Integrated Workflow	✗	✗	✗	✗	✗	✓

Table 1.1: Gap analysis of VibeShift against existing platforms.

The analysis identifies four critical gaps VibeShift uniquely fills: AI genre transformation on mobile, an integrated stem-to-genre workflow, vocal preservation during transformation, and per-stem timbre editing — none of which exist in any single surveyed platform.

1.7 Document Organisation

The remainder of this document is structured as follows: Chapter 2 reviews related academic literature and existing commercial solutions. Chapter 3 presents the complete requirements specification. Chapter 4 describes the system architecture, database design, and application design artefacts. Chapter 5 details the full implementation. Chapter 6 presents testing and evaluation results. Chapter 7 concludes the report with a summary of achievements, challenges, and future directions.

2.0 Related Work

2.1 Chapter Overview

This chapter reviews the academic literature and existing commercial solutions most relevant to VibeShift's two core technical domains: audio source separation and AI-based genre transformation. It concludes with a comparative analysis that positions VibeShift within the current landscape and justifies the principal technical choices made during development.

2.2 Audio Source Separation Techniques

Audio source separation — isolating individual sound sources from a mixed audio signal — is one of the longest-standing problems in audio signal processing. Research has progressed through three generations: classical signal processing, matrix factorisation, and deep neural network approaches.

Early methods such as Independent Component Analysis (ICA) [5] assumed statistical independence between sources, performing adequately for simple instantaneous mixtures but failing under the convolutive mixing conditions present in real-world music recordings. Non-Negative Matrix Factorisation (NMF) [7] decomposed mixture spectrograms into instrument-specific basis vectors and temporal activations, successfully applied to drum transcription and vocal separation [8], but requiring the number of sources to be pre-specified and discarding phase information.

Deep learning transformed the field from approximately 2014. Jansson et al. [11] demonstrated that a U-Net architecture effectively separates singing voice from accompaniment using hierarchical spectro-temporal features. Wave-U-Net [12] extended this to the waveform domain, eliminating the phase reconstruction problem. Demucs [1] employs a time-domain encoder-decoder architecture with U-Net skip connections trained on the MUSDB18 dataset [15]. Hybrid Demucs [14] achieves a state-of-the-art average SDR of 9.00 dB across four stems — the highest among open-source systems.

VibeShift uses Hybrid Demucs as its stem separation backbone, selected over Spleeter [16] (6.9 dB SDR) and Open-Unmix [17] (5.3 dB SDR) on grounds of superior benchmark performance, clean Python API integration, and robust handling of arbitrary-length audio. HiFi-GAN [20] was adopted for mel-to-audio resynthesis within the CycleGAN pipeline for its balance of synthesis quality and inference efficiency on the Google Colab T4 GPU.

2.3 AI-Based Genre Transfer and Style Transfer

2.3.1 GAN-Based Spectrogram Translation

Zhu et al.'s CycleGAN [26] introduced cycle-consistent adversarial learning for unpaired image-to-image translation. The cycle consistency loss enforces that translating from domain A to domain B and back reconstructs the original input. Brunner et al. [27] applied CycleGAN to symbolic music for genre transfer, demonstrating cycle consistency effectively preserves melodic structure while modifying style. VibeShift's initial genre transformation module implements this as

a 2D convolutional encoder-decoder with six residual blocks and Instance Normalisation, paired with a PatchGAN discriminator [25], trained on GTZAN mel-spectrograms.

Despite producing reasonable results for closely related genre pairs, fundamental limitations emerged: the L1 cycle consistency loss causes over-smoothing [29]; the model cannot add or remove instruments; and the per-pair training overhead of 48 hours per pair on Google Colab T4 makes scaling to all 45 directional pairs impractical.

2.3.2 Generative Audio Models

MusicGen [2], introduced by Copet et al. at Meta AI in 2023, is a single-stage auto-regressive Transformer trained on 20,000 hours of licensed music. It uses EnCodec [35] as its audio tokeniser and generates sequences conditioned on text descriptions from a T5 encoder [36]. MusicGen-Melody accepts a melody conditioning signal from an input audio clip, generating music that follows the melodic contour while adopting the target style. This melody conditioning is central to VibeShift's production pipeline. MusicGen was selected over Riffusion [37] and AudioLDM [40] on the basis of native melody conditioning and availability on the Replicate cloud inference platform.

2.4 Existing Music Editing Platforms

Professional DAWs — Ableton Live, Logic Pro, Pro Tools — offer gold-standard music production but are expensive (\$199–\$749), computationally demanding, and require significant expertise. LALAL.AI offers commercial stem separation via a freemium mobile app but provides no genre transformation or timbre editing. Spleeter [16] is a research-grade open-source CLI tool with no mobile application. Moises App is the closest commercial competitor, offering stem separation and basic pitch/tempo control, but lacking genre transformation and timbre editing. AI generation platforms — Soundraw and Suno AI — generate new music from textual descriptions but cannot transform existing user-uploaded audio.

2.5 Comparative Analysis

Model	Architecture	MUSDB18 SDR	Open Source	Used In
Open-Unmix [17]	BiLSTM	5.3 dB	Yes	Baseline
Spleeter [16]	U-Net (spectrogram)	6.9 dB	Yes	Baseline
Hybrid Demucs [14]	Hybrid time+freq	9.0 dB	Yes	VibeShift

Table 2.1: Comparison of source separation models on MUSDB18.

Approach	Training	Genres	Quality	Vocal Pres.	Scalability
CycleGAN (v1)	48h/pair	3 pairs	6/10	No	Poor
CAUNet [29]	72h/pair	Custom	7/10	No	Poor

Approach	Training	Genres	Quality	Vocal Pres.	Scalability
MusicGen [2] (v2)	None	Unlimited	9/10	Yes	Excellent

Table 2.2: Comparative evaluation of genre transformation approaches.

2.6 Positioning VibeShift

VibeShift occupies a unique position — neither a single-algorithm research prototype nor a professional DAW, but a production-quality mobile application making state-of-the-art research accessible to non-technical users. Four properties define its distinctive contribution:

- **Research-to-Application Translation:** Operationalises Hybrid Demucs, CycleGAN, and MusicGen within a single integrated mobile workflow.
- **Documented Model Evolution:** The transition from CycleGAN to MusicGen constitutes an empirical comparative evaluation of two fundamentally different genre transformation paradigms.
- **Vocal Preservation Architecture:** The multi-stage pipeline — Demucs separation, genre FX, MusicGen melody conditioning, BPM matching, parametric mixing — is absent from all reviewed platforms.
- **Cloud-Native Mobile Architecture:** The combination of React Native, FastAPI, Clerk, AWS S3, and Replicate API is a novel architectural pattern for AI-powered mobile audio not documented in the music technology literature.

3.0 Requirements

3.1 Chapter Overview

This chapter presents the complete requirements specification for VibeShift, covering functional requirements, non-functional requirements, structured use cases for the five core workflows, and system constraints and assumptions that governed the design.

3.2 Functional Requirements

3.2.1 User Account and Authentication

Users can register and log in using email and password or via Google and GitHub OAuth, both managed through the Clerk authentication service. On first login, a user record is automatically created in the VibeShift database via POST /auth/sync. All API requests are authenticated using Clerk-issued RS256 JWT tokens verified against Clerk's public JWKS endpoint. Users can view and update their profile including name, bio, avatar, audio quality preference, and export format. The profile screen displays aggregated usage statistics — total tracks demixed and total genre transformations completed.

3.2.2 Audio Upload and Management

Authenticated users can upload audio files in MP3, WAV, M4A, and FLAC formats. All uploaded and processed files are stored in AWS S3 under user-specific key paths. The mobile client retrieves files via time-limited presigned S3 URLs. The Library screen displays the user's full processing history with three filterable tabs: All, Demixed, and Transform. The Home screen displays the five most recently processed tracks under Recently Edited.

3.2.3 Stem Separation (Demixing)

The system separates uploaded audio into four stems — vocals, drums, bass, and other — using Demucs. Separation runs as an asynchronous background task returning a job ID immediately. The job lifecycle passes through four states: pending, processing, completed, and failed, persisted in the demix_jobs table. A polling endpoint (GET /demix/{job_id}/status) returns the current state and presigned S3 URLs for all four stems on completion. Mono audio is automatically converted to stereo before Demucs processing.

3.2.4 Interactive Stem Editing

Users can independently edit any separated stem with three controls: pitch shifting (−12 to +12 semitones via the librosa phase vocoder), timbre modification (0.0 to 2.0 strength via HPSS harmonic enhancement), and volume adjustment (linear multiplier). Individual stems can be muted. Users can add custom stems by uploading additional audio files, automatically padded to match the original song length. All editing parameters are persisted in the stems table.

3.2.5 Genre Transformation

The system transforms uploaded audio into any of 10 supported genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. Transformation runs asynchronously through a multi-stage pipeline: (a) Demucs vocal/instrumental separation; (b) genre-specific vocal FX; (c) instrumental resampling to 32,000 Hz; (d) MusicGen-Melody generation via Replicate API; (e) BPM-based tempo matching; (f) parametric vocal and instrumental mixing. Users may also provide a custom freeform text prompt. Job state and all parameters are persisted in the `transform_jobs` table.

3.2.6 Project History and Export

Users can export the final mixed audio from any demix session by running the mixer pipeline (`pad_and_sum` with peak normalisation), saving to S3 and recording in `demix_mixes`. All completed outputs — stems, mixes, and genre-transformed audio — can be downloaded via presigned S3 URLs. All job and file records are retained indefinitely in the Library screen.

3.3 Non-Functional Requirements

3.3.1 Performance

All non-ML API endpoints shall respond within 500 milliseconds. Stem separation shall complete within 3 minutes for a standard 3–5 minute audio file. Genre transformation shall complete within 90 seconds for a 35-second clip. The mobile app shall display a processing indicator within 1 second of job submission and update within 5 seconds via polling.

3.3.2 Security

All endpoints require a valid Clerk RS256 JWT Bearer token; invalid tokens receive HTTP 401. JWT verification uses JWKS public key caching with no secret key on the backend. All S3 buckets use private permissions with time-limited presigned URL access only. All traffic is encrypted via HTTPS/TLS. Data access is strictly scoped by authenticated user ID.

3.3.3 Usability

The application requires no technical setup to operate. All asynchronous operations display clear progress indicators and descriptive status messages. Errors are communicated via user-facing messages, not raw HTTP codes. The app supports iOS and Android from a single React Native codebase with a consistent dark-themed UI.

3.3.4 Scalability

The Dockerised backend supports horizontal scaling without shared local file system state. AWS S3 scales independently of backend compute. MusicGen inference is offloaded to Replicate API, enabling unlimited parallel requests without on-premises GPU capacity.

3.3.5 Reliability

All job states are persisted in the database, surviving backend restarts. Failed jobs record error messages for diagnosis. A health check endpoint (`GET /health`) exposes service status and model availability. Replicate API timeouts are handled gracefully, marking jobs as failed rather than leaving them in a perpetual processing state.

3.4 Use Cases

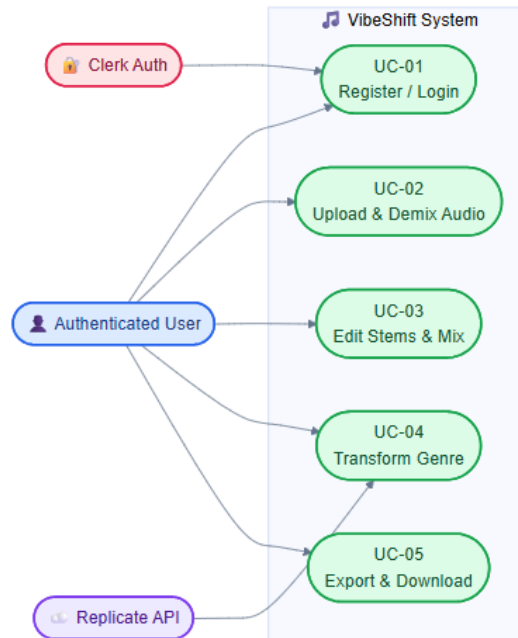


Figure 3.1 — VibeShift Use Case Diagram.

3.4.1 User Registration and Authentication: A new user opens the app, selects Sign Up, enters credentials, and is registered via Clerk. POST /auth/sync creates their database record. The access token is stored in AsyncStorage and attached to all future requests. Alternatively, the user logs in via Google or GitHub OAuth through Clerk, following the same database sync flow.

3.4.2 Audio Upload and Demixing: The user taps Upload Song, selects an audio file, and the app uploads it to S3. A demix job is submitted and a job ID returned immediately. The app polls the status endpoint every 3 seconds. The backend converts audio to stereo, runs Demucs, uploads all four stems to S3, and marks the job completed. The app displays the stems with editing controls.

3.4.3 Stem Editing and Mixing: The user opens a completed demix job and adjusts individual stems using pitch, timbre, and volume controls. Each edit triggers a backend call that processes the modification and saves the updated stem to S3. The user mutes unwanted stems, then taps Mix and Export.

3.4.4 Genre Transformation: The user uploads a song, selects a target genre from the bubble selector, and submits the job. The backend runs the full transformation pipeline as a background task. The app polls until completion and presents the transformed audio for playback and download.

3.4.5 Export and Download: The user navigates to the Library screen, selects a filter tab (All, Demixed, or Transform), and taps a completed track. The app requests a fresh presigned S3 URL and downloads the audio file directly from S3 to the device's local storage.

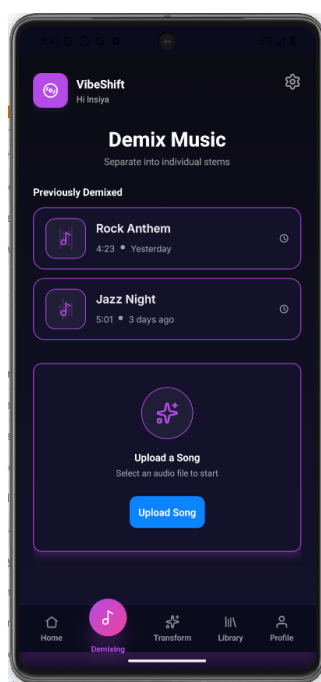


Figure 3.2 — VibeShift Demix Music Screen.

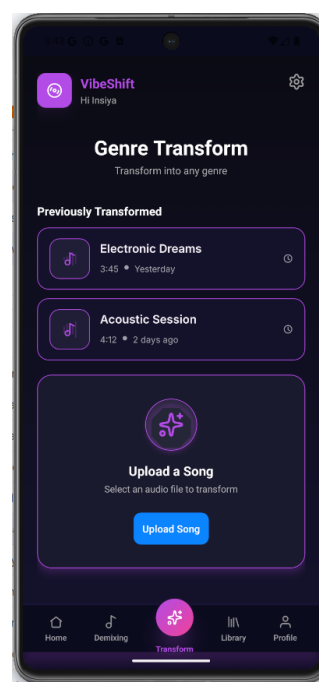


Figure 3.3 — VibeShift Genre Transform Screen.

3.5 System Constraints and Assumptions

Constraints:

- Demucs requires stereo input; mono files are auto-converted before processing.
- MusicGen operates at 32,000 Hz; all input audio is resampled before melody conditioning.
- MusicGen via Replicate supports a maximum of 60 seconds output per inference call.
- All audio processing is server-side; the mobile client performs no local computation.
- The database uses SQLite in development, migrating to PostgreSQL via DATABASE_URL with no code changes.

Assumptions:

- Users have a stable internet connection (minimum 1 Mbps) for upload and download.
- Uploaded audio is of reasonable quality (minimum 128 kbps MP3) for meaningful processing results.
- Users have the right to process uploaded audio — no copyright verification is performed.
- Clerk, AWS S3, and Replicate API are correctly configured and available.

4.0 Design

4.1 Chapter Overview

This chapter presents the complete system design of VibeShift across four dimensions: system architecture, database design, application-level design artefacts, and UI/UX design. The architecture section describes how components are organised and communicate. The database section defines the schema supporting both processing modules. The application design section presents sequence diagrams, state diagrams, data flow diagrams, and the class diagram. The UI/UX section describes design principles and key screens.

4.2 System Architecture

4.2.1 High-Level Architecture

VibeShift follows a client-server architecture with clear separation between the mobile frontend, backend API layer, AI processing layer, and cloud storage. The mobile application communicates exclusively with the FastAPI backend over HTTPS. The backend orchestrates all processing — authenticating requests, managing database state, delegating ML computation, and managing S3 file storage. The mobile client never communicates directly with AI services.

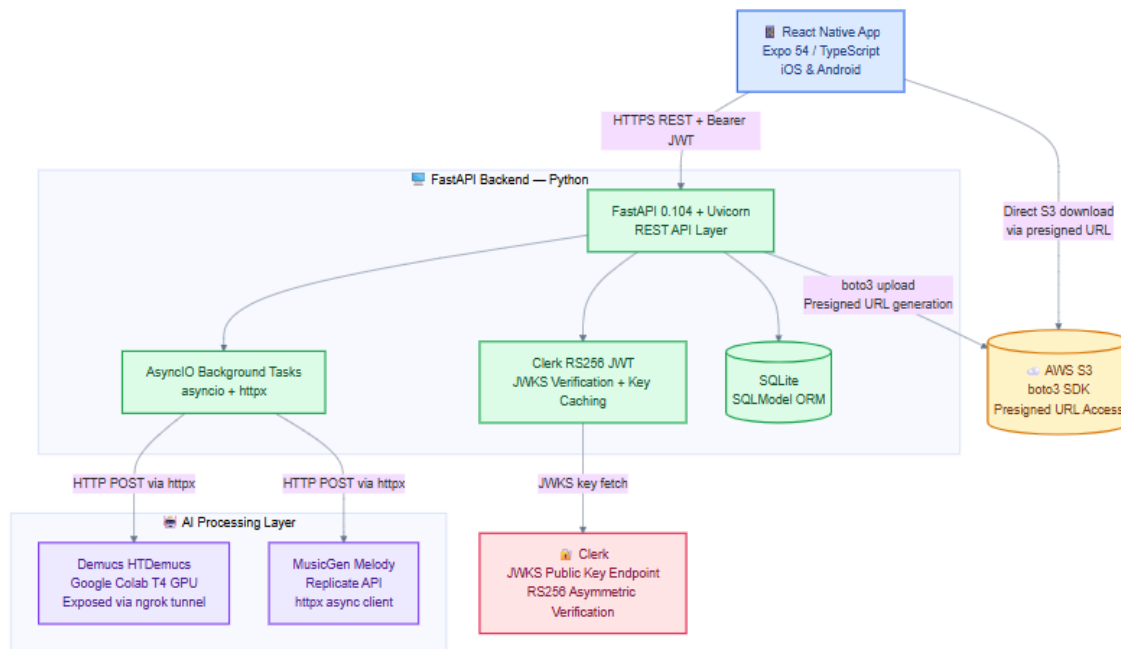


Figure 4.1 — VibeShift High-Level System Architecture.

The five principal components are: React Native Frontend (Expo 54, TypeScript, iOS and Android); FastAPI Backend (FastAPI 0.104 + Uvicorn, five route groups, SQLite via SQLModel, AsyncIO background tasks); AI Processing Layer (Demucs on Google Colab T4 via ngrok tunnel; MusicGen on Replicate via httpx); AWS S3 (private bucket, boto3 upload, presigned URL download); and

Clerk Authentication (RS256 JWT, JWKS public key caching).

4.2.2 Microservices Architecture

Within the backend, five routers map to five feature areas: auth, demix, transform, library, and user. The ML service layer comprises four stateless modules: DemucsService, MusicGenService, EditorService, and MixerService, called exclusively from AsyncIO background tasks.

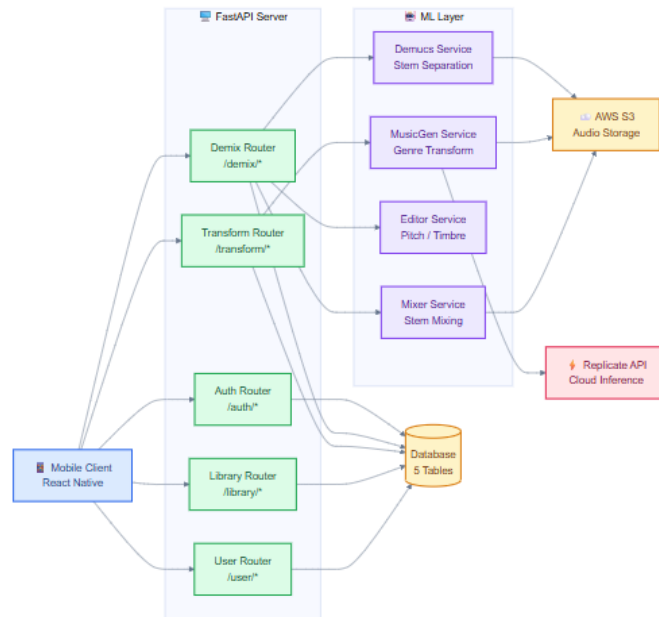


Figure 4.2 — VibeShift Microservices Architecture.

4.2.3 Deployment Architecture

The FastAPI backend is packaged as a Docker image and deployed to an AWS EC2 instance provisioned via AWS CloudFormation. Audio files are stored in AWS S3. MusicGen inference is offloaded to the Replicate API. Demucs inference runs on a Google Colab T4 GPU exposed via ngrok.

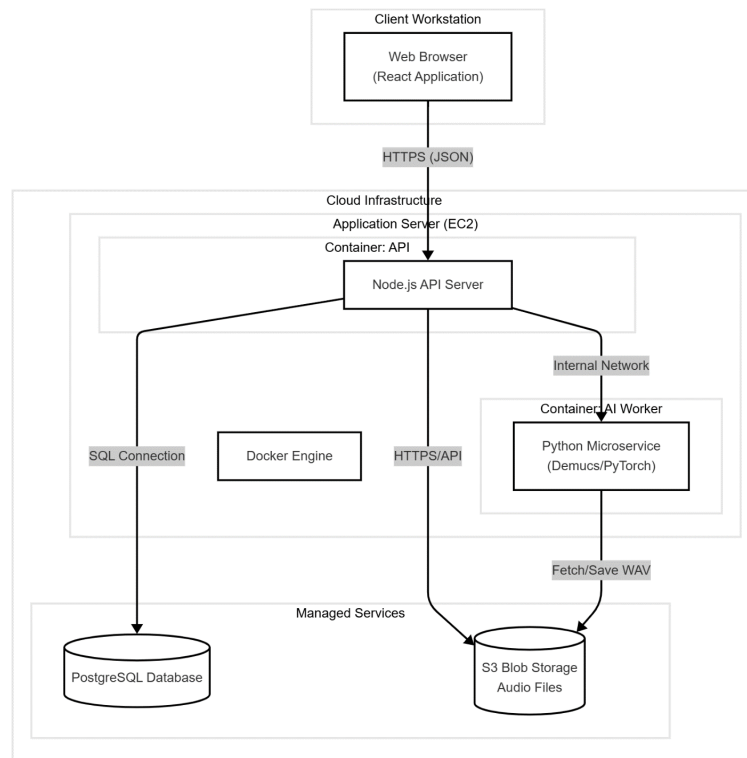


Figure 4.3 — VibeShift Deployment Architecture.

4.3 Database Design

4.3.1 ER Diagram

The VibeShift database comprises five tables: `users` (one-to-many with `demix_jobs` and `transform_jobs`), `demix_jobs` (one-to-many with `stems` and `demix_mixes`). All primary keys are UUIDs. All timestamps default to UTC.

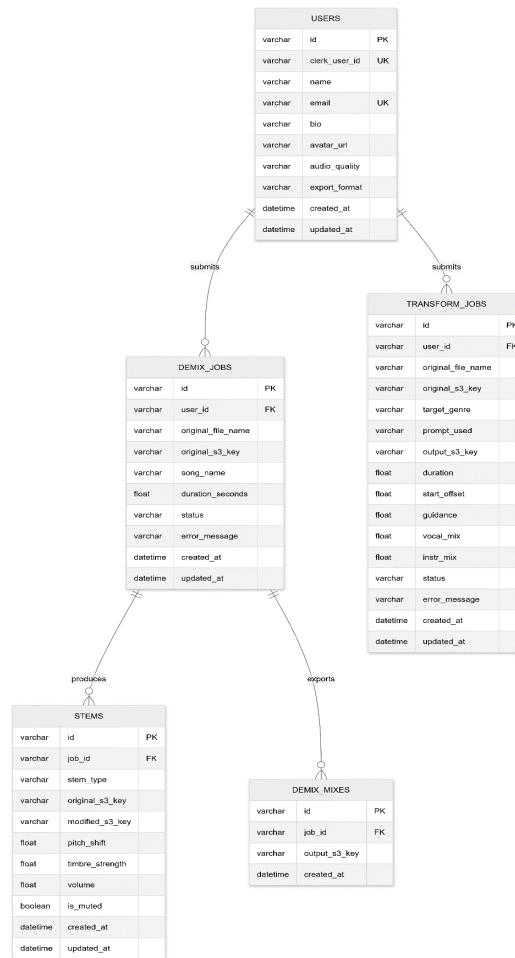


Figure 4.4 — VibeShift Entity Relationship Diagram.

4.3.2 Schema Design

users: Stores one row per registered user. `clerk_user_id` is the stable Clerk identifier. `audio_quality` and `export_format` store user preferences.

demix_jobs: Tracks stem separation job lifecycle. The status column (pending → processing → completed → failed) is polled by the mobile client.

stems: One row per stem per demix job. `pitch_shift`, `timbre_strength`, `volume`, and `is_muted` persist the user's editing state.

demix_mixes: Records each mix export. A user may export multiple mixes from the same job with different stem settings.

transform_jobs: Tracks genre transformation jobs including `target_genre`, `prompt_used`, MusicGen inference parameters, and `output_s3_key`.

4.3.3 Data Dictionary

Table	Column	Type	Description
users	clerk_user_id	VARCHAR UNIQUE	Clerk stable user identifier
users	audio_quality	VARCHAR	low / standard / high

Table	Column	Type	Description
users	export_format	VARCHAR	mp3 / wav / flac
demix_jobs	status	VARCHAR	pending / processing / completed / failed
demix_jobs	original_s3_key	VARCHAR	S3 object path of uploaded audio
demix_jobs	duration_seconds	FLOAT	Audio length populated by Demucs on completion
stems	stem_type	VARCHAR	vocals / drums / bass / other
stems	pitch_shift	FLOAT	Semitone offset: -12 to +12
stems	timbre_strength	FLOAT	HPSS harmonic blend: 0.0 to 2.0
stems	is_muted	BOOLEAN	Exclude stem from mix when True
transform_jobs	target_genre	VARCHAR	Selected genre preset
transform_jobs	prompt_used	VARCHAR	Resolved MusicGen text prompt
transform_jobs	guidance	FLOAT	CFG scale — default 9.5
transform_jobs	vocal_mix	FLOAT	Vocal loudness multiplier
transform_jobs	instr_mix	FLOAT	Instrumental loudness multiplier

Table 4.1 — VibeShift Data Dictionary (key columns).

4.4 Application Design

4.4.1 Sequence Diagrams

4.4.1.1 User Authentication Sequence: The authentication sequence begins when the user enters credentials. The React Native app passes them to Clerk, which returns an RS256 JWT. The app sends this to POST /auth/sync. The backend verifies the token against Clerk's JWKS endpoint, creates a user record if none exists, and returns the full user profile. The token is stored in AsyncStorage for all subsequent requests.

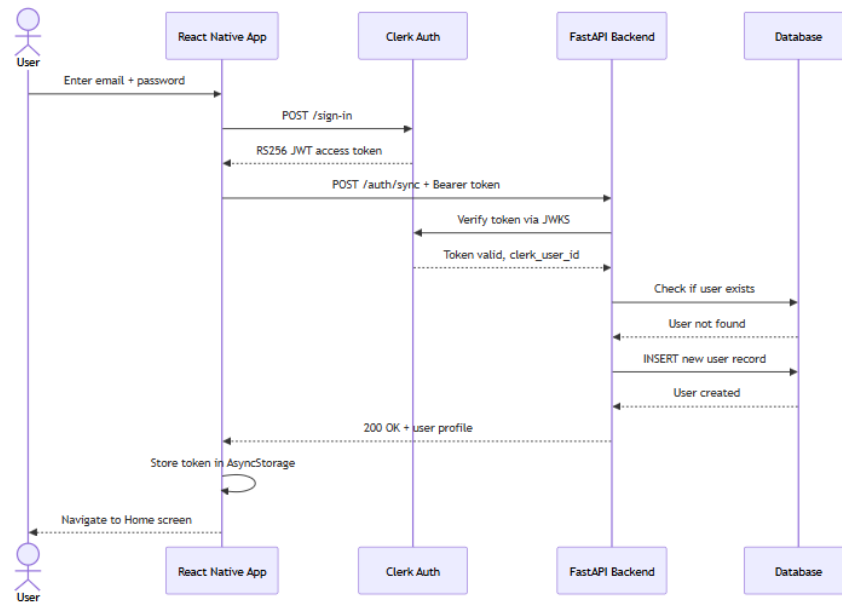


Figure 4.5 — User Authentication Sequence Diagram.

4.4.1.2 Audio Upload and Demixing Sequence: The user selects a file, the app uploads it to S3, and submits a demix job. The backend returns a job ID and launches an AsyncIO background task that runs Demucs, uploads four stems to S3, and updates the database. The mobile client polls every 3 seconds and receives presigned stem URLs on completion.

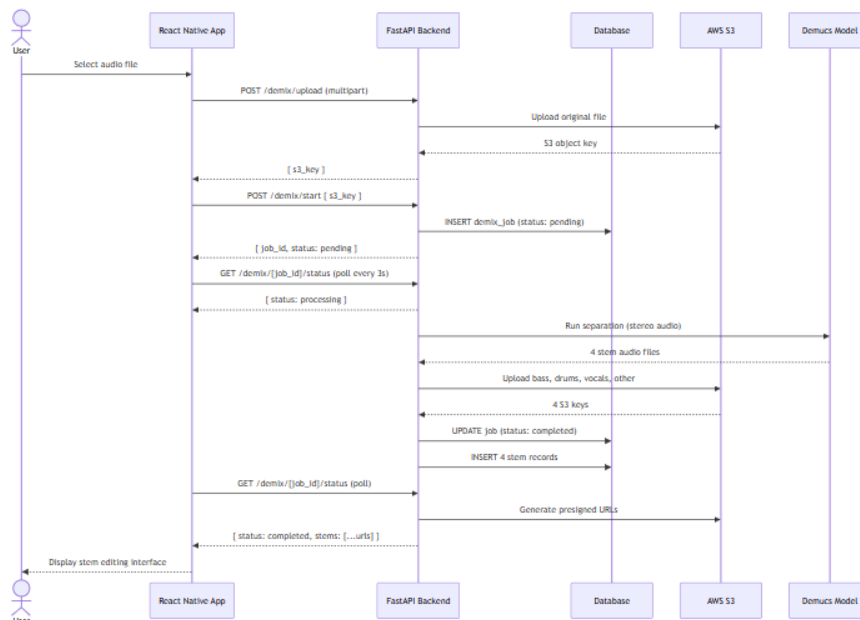


Figure 4.6 — Audio Upload and Demixing Sequence Diagram.

4.4.1.3 Genre Transformation Sequence: The user selects a file and genre and submits the job. The backend returns a job ID and starts the multi-stage pipeline: Demucs vocal separation, genre

FX, MusicGen via Replicate, BPM tempo matching, vocal and instrumental mixing, S3 upload, and job completion. The app polls until completion and presents the output.

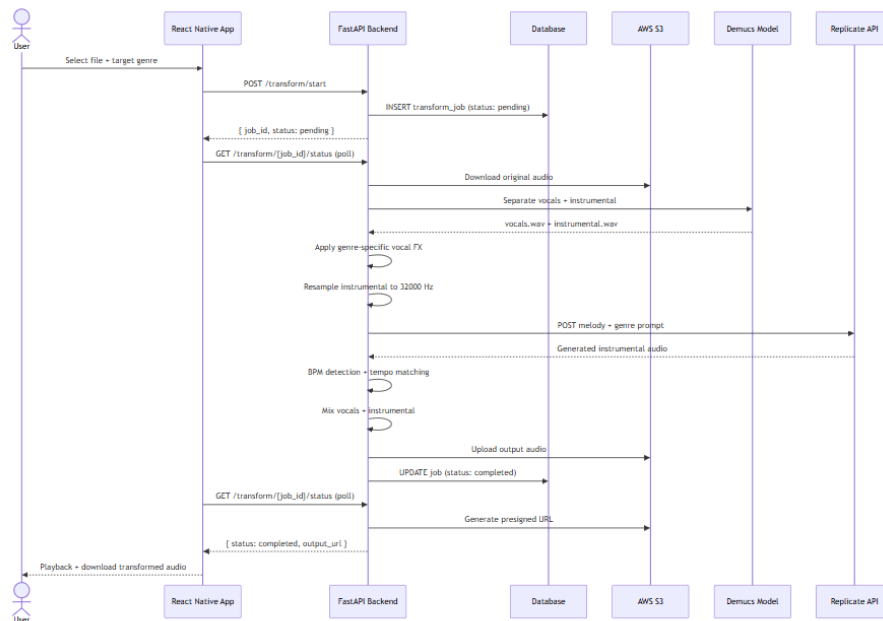


Figure 4.7 — Genre Transformation Sequence Diagram.

4.4.2 State Diagrams

4.4.2.1 Audio Processing State Diagram: Both demix and transform jobs share a four-state lifecycle: Pending (on submission), Processing (when the ML pipeline begins), Completed (on successful S3 upload), and Failed (on any unhandled exception or API timeout, with error message recorded in the database).

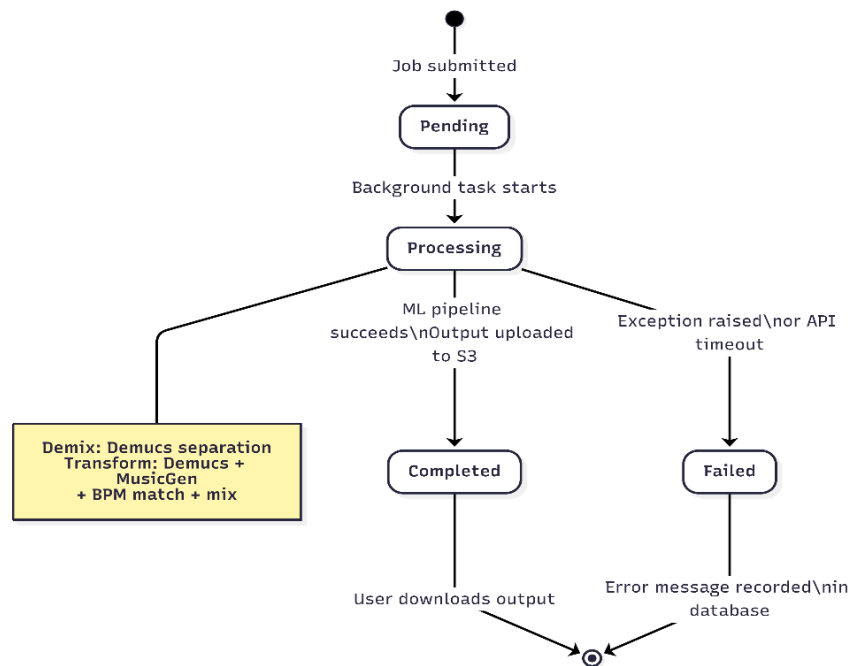


Figure 4.8 — Audio Processing Job State Diagram.

4.4.2.2 User Session State Diagram: The session has two states: Unauthenticated and Authenticated. On launch, the app checks AsyncStorage for a stored JWT. A valid token transitions to Authenticated (Home screen). No token or HTTP 401 keeps the session Unauthenticated (Login screen). Logout clears the token and returns to Unauthenticated.

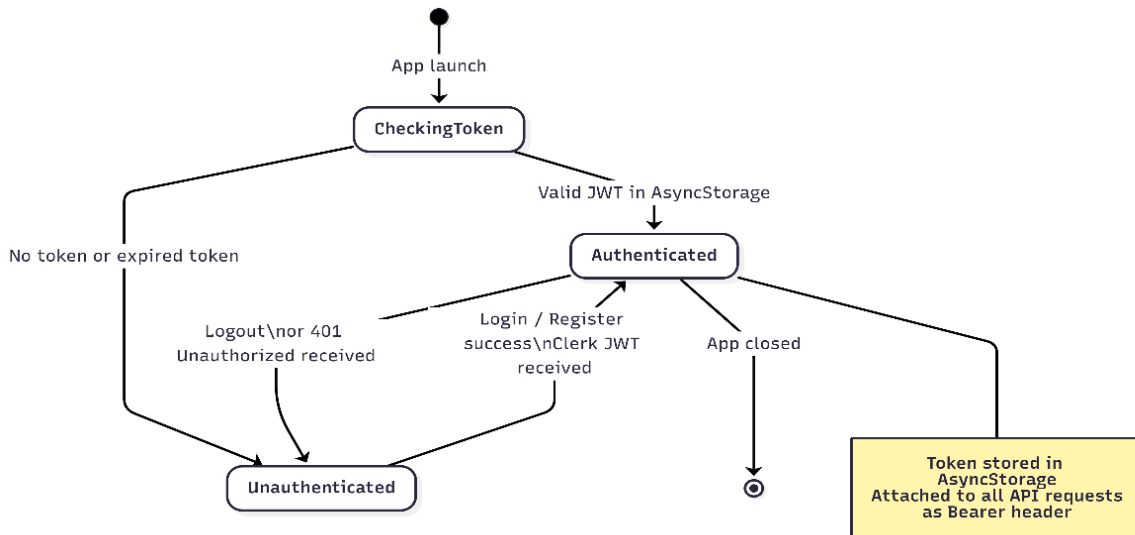


Figure 4.9 — User Session State Diagram.

4.4.3 Data Flow Diagram

The Level 0 DFD presents VibeShift as a single system interacting with three external entities: the User (audio input / processed audio output), AWS S3 (audio storage), and External AI Services (Demucs via ngrok, MusicGen via Replicate).

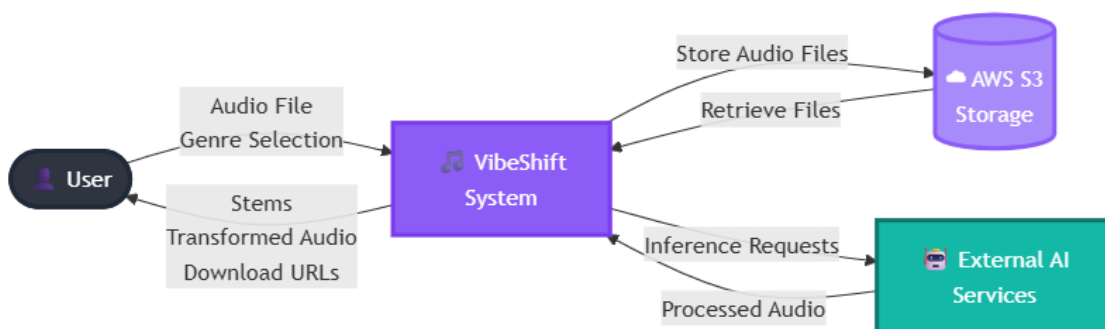


Figure 4.10 — Data Flow Diagram Level 0.

The Level 1 DFD decomposes the system into three processes: Authentication (1.0) — Clerk JWT verification and user database sync; Stem Separation (2.0) — upload, separate, edit, mix workflow; Genre Transformation (3.0) — MusicGen pipeline orchestration.

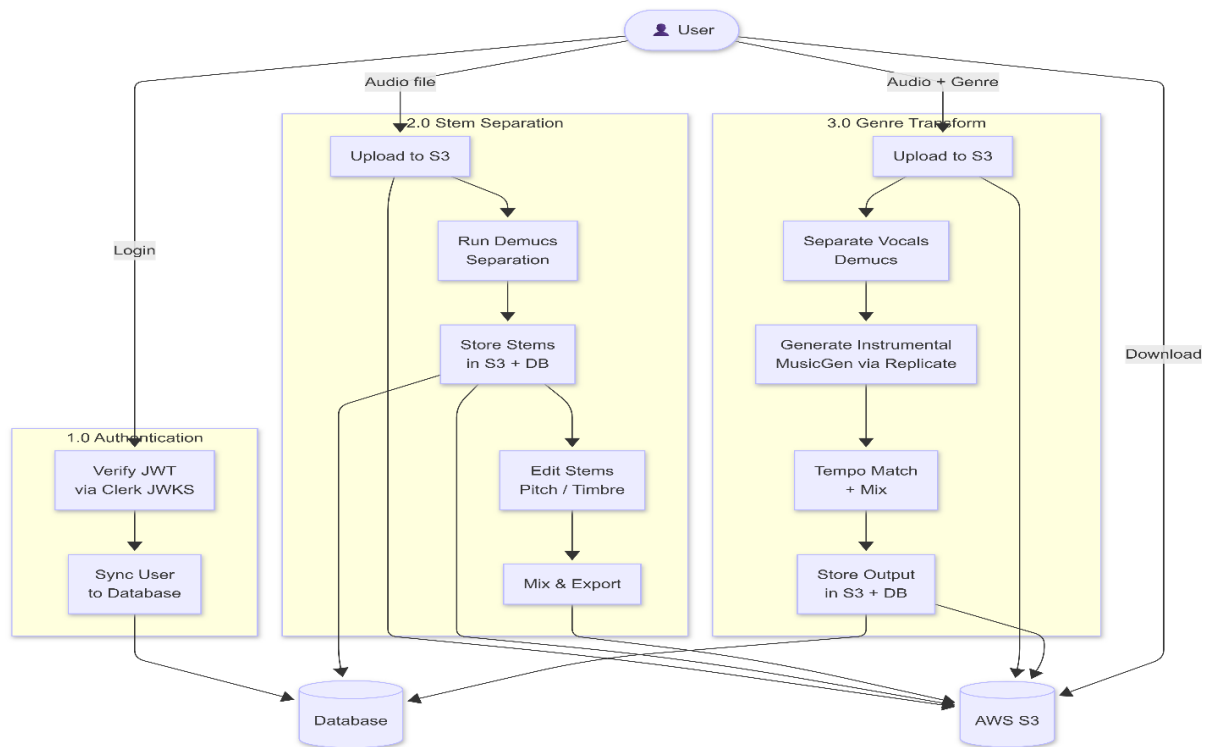


Figure 4.11 — Data Flow Diagram Level 1.

4.4.4 Class Diagram

The class diagram shows five SQLAlchemy database entities and three ML service classes. The three service classes — `DemucsService`, `MusicGenService`, and `MixerService` — are stateless, making them safe for AsyncIO concurrent background tasks.

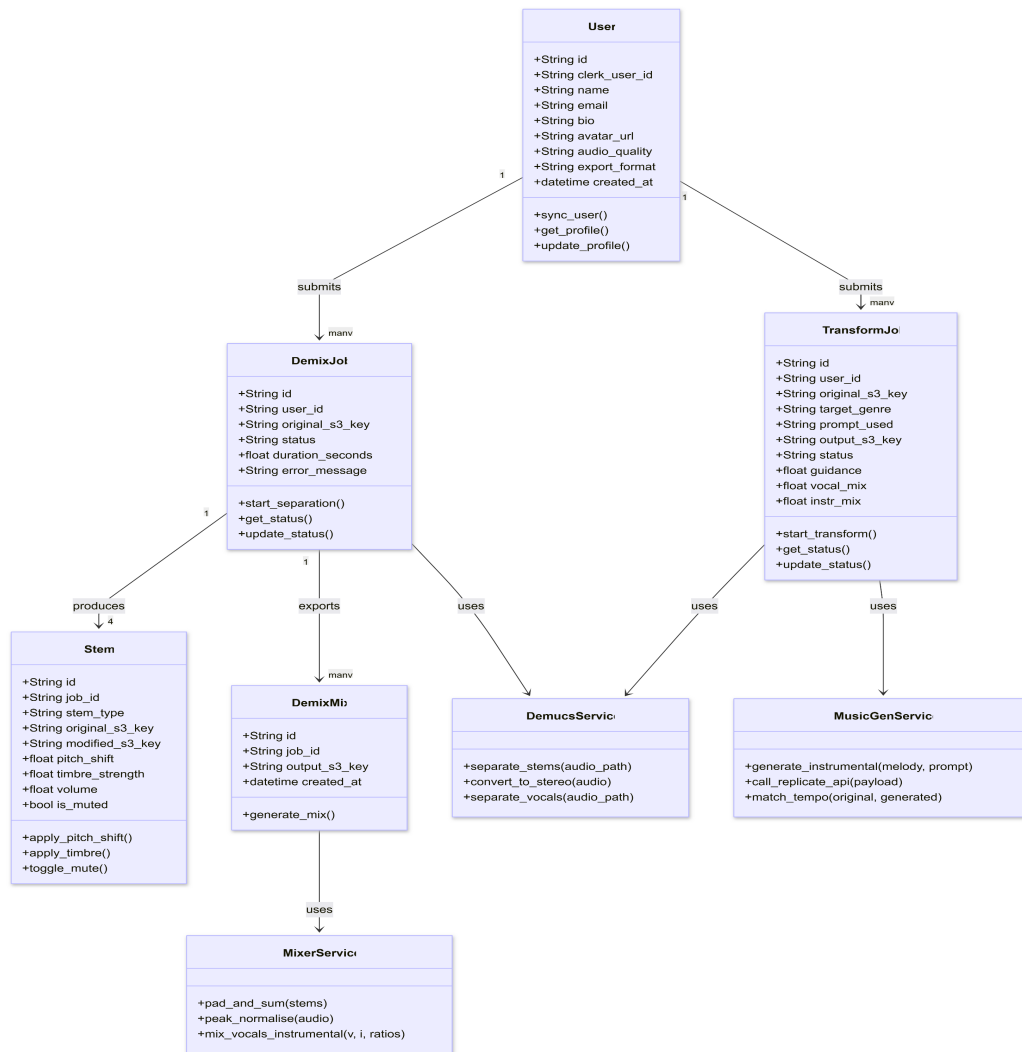


Figure 4.12 — VibeShift Backend Class Diagram.

4.5 UI/UX Design

4.5.1 Design Principles

- **Simplicity over Feature Density:** Each screen exposes exactly one primary action; complex controls are revealed progressively after processing completes.
- **Dark Theme for Audio Context:** Dark background with purple and teal accents, standard in professional audio applications, reducing eye strain during extended listening sessions.
- **Status Transparency:** All asynchronous operations display explicit human-readable status messages during ML processing.
- **Card-Based Information Architecture:** All track listings use a consistent SongCard component for a familiar, scannable interface.
- **Bottom Tab Navigation:** Five sections — Home, Demixing, Transform, Library, Profile — accessible via a persistent bottom tab bar following iOS and Android conventions.

4.5.2 Key Screens

The following figures display the primary screens of the VibeShift mobile application. UI screenshots are presented in pairs for space efficiency.

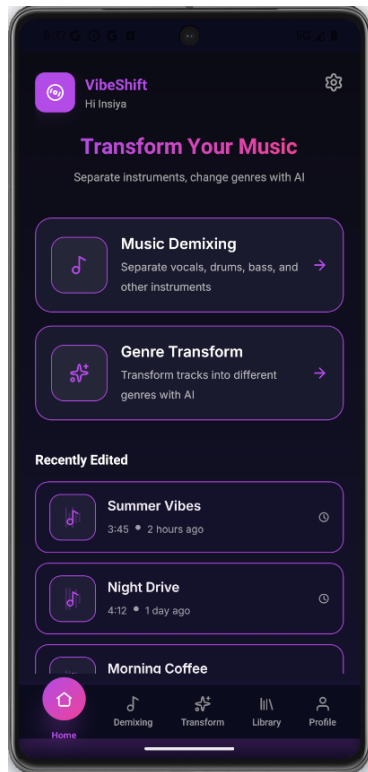


Figure 4.13 — VibeShift Home Screen.

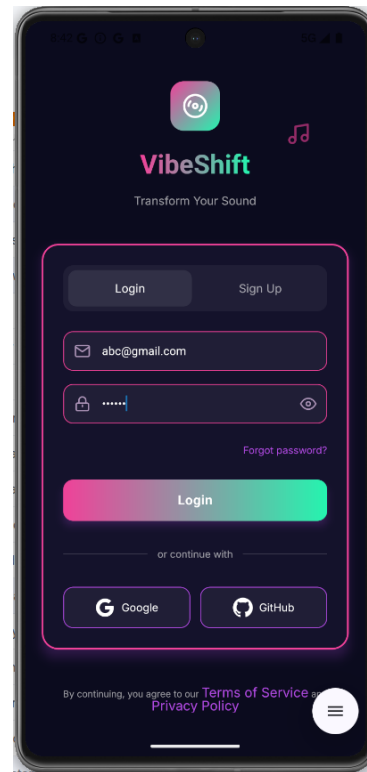


Figure 4.14 — VibeShift Login Screen.

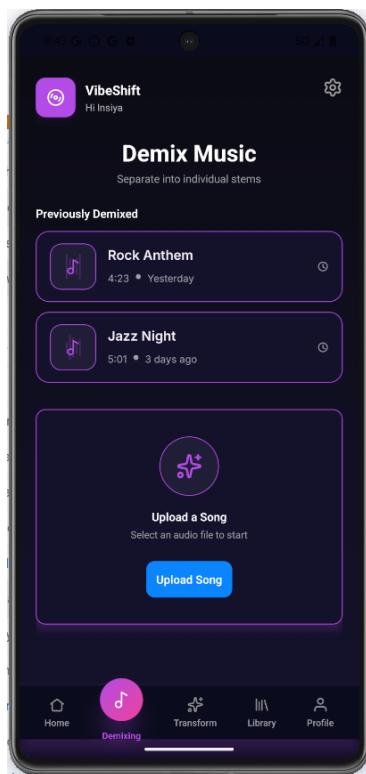


Figure 4.15 — VibeShift Demix Music Screen.

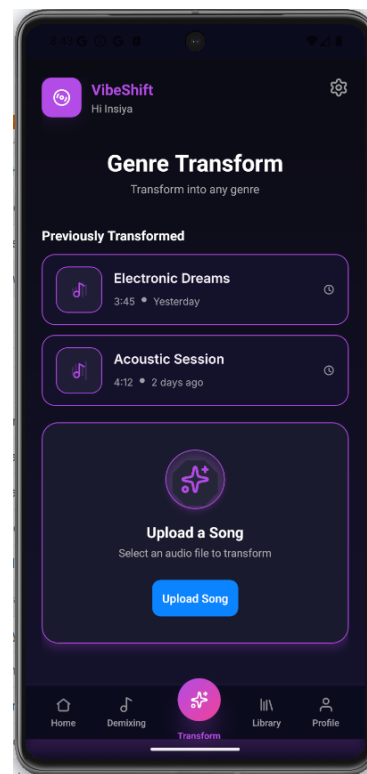


Figure 4.16 — VibeShift Genre Transform Screen.

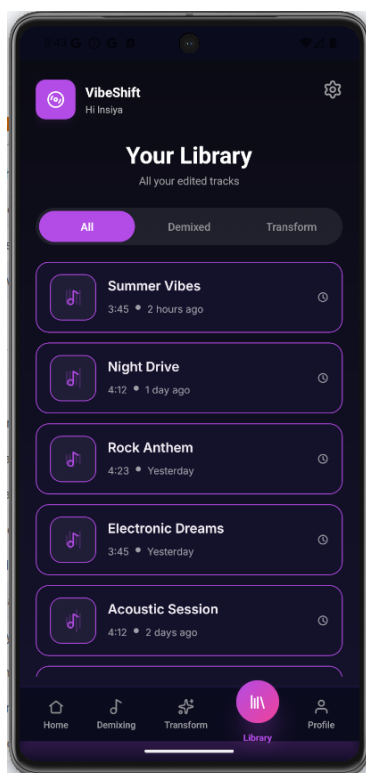


Figure 4.17 — VibeShift Library Screen.

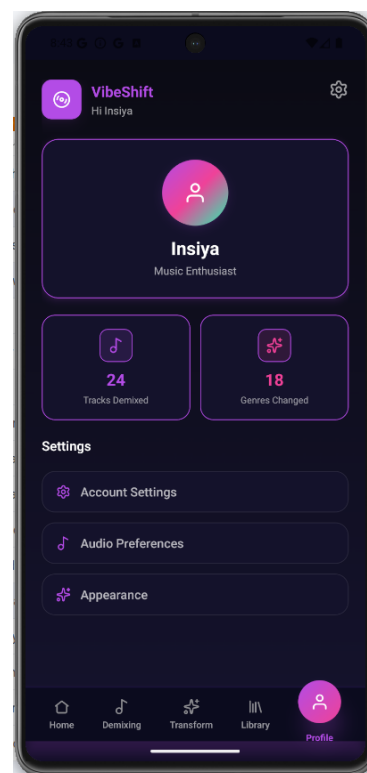


Figure 4.18 — VibeShift Profile Screen.

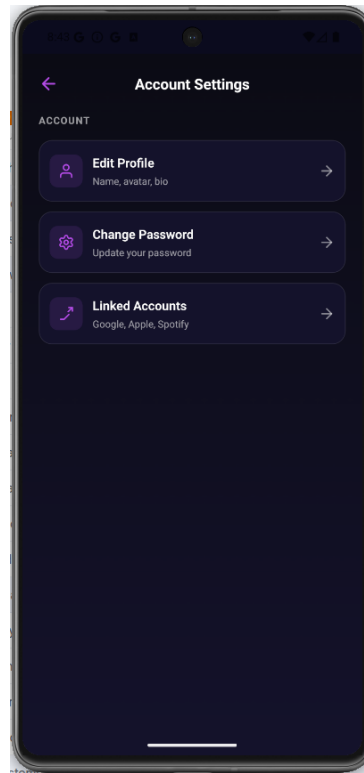


Figure 4.19 — VibeShift Account Settings Screen.

4.6 Key Code Snippets

This section presents the key implementation code snippets that underpin the VibeShift system design. These snippets illustrate the core algorithmic and architectural decisions described in the preceding sections.

4.6.1 FastAPI Backend — Health Check and Initialisation

The following code shows the FastAPI application initialisation including CORS middleware configuration and the Replicate model identifiers for Demucs and MusicGen:

```
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
import replicate, tempfile, os

app = FastAPI(title='VibeShift API', version='1.0.0')
app.add_middleware(CORSMiddleware, allow_origins=['*'],
allow_methods=['*'], allow_headers=['*'])

DEMUCS_MODEL = 'cjbwbw/demucs:25a173108cff36ef9f80f854c162d01df9e6528be175794b81158fa03836d953'
MUSICGEN_MODEL = 'meta/musicgen:7a76a8258b23fae65c5a22debb8841d1d7e816b75c2f24218cd2bd8573787906'

@app.get('/health')
def health():
    token = os.environ.get('REPLICATE_API_TOKEN', '')
    return {'status': 'ok', 'replicate_token_set': bool(token)}
```

Code Snippet 4.1 — FastAPI Application Initialisation.

4.6.2 Demix Endpoint — Stem Separation

The /demix endpoint accepts an uploaded audio file, saves it to a temporary location, and calls the Demucs model on Replicate to produce four stem URLs:

```
@app.post('/demix')
async def demix(file: UploadFile = File(...), model_name: str = Form(default='htdemucs')):
    suffix = os.path.splitext(file.filename)[-1] or '.mp3'
    with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
        tmp.write(await
```

```
file.read()) tmp_path = tmp.name try: with open(tmp_path, 'rb') as f: output =
replicate.run(DEMUCS_MODEL, input={'audio': f, 'model': model_name, 'float32': True,
'mp3_bitrate': 320}) return output # {'vocals': url, 'drums': url, 'bass': url, 'other':
url} finally: os.unlink(tmp_path)
```

Code Snippet 4.2 — Demix Endpoint (Stem Separation via Replicate).

4.6.3 Genre Transform Endpoint — MusicGen Integration

The /genre-transform endpoint implements the genre-specific prompt lookup table and calls MusicGen via the Replicate API with melody conditioning:

```
GENRE_PROMPTS = { 'blues': 'blues music, slide guitar, harmonica, soulful vocals, slow
rhythm', 'classical': 'classical orchestral music, strings, piano, symphony, elegant',
'country': 'country music, acoustic guitar, banjo, twang, fiddle, storytelling', 'disco':
'disco music, funky bass guitar, four-on-the-floor drums, synthesizer', 'hiphop': 'hip
hop music, boom bap drums, heavy bass, rap, urban, drum machine', 'jazz': 'jazz music,
piano, upright bass, trumpet, swing, improvisation', 'metal': 'heavy metal, distorted
electric guitar, double bass drum, aggressive', 'pop': 'pop music, catchy hook,
synthesizer, polished production, upbeat', 'reggae': 'reggae music, offbeat guitar, bass,
relaxed, jamaican, one drop', 'rock': 'rock music, electric guitar, live drums, bass,
energetic, band', } @app.post('/genre-transform') async def genre_transform(file:
UploadFile = File(...), target_genre: str = Form(...), duration: float =
Form(default=10.0), guidance: float = Form(default=9.5)): prompt =
GENRE_PROMPTS.get(target_genre, target_genre) with
tempfile.NamedTemporaryFile(delete=False) as tmp: tmp.write(await file.read()) tmp_path =
tmp.name try: vocal_stem = _demix_audio(tmp_path) generated =
replicate.run(MUSICGEN_MODEL, input={'model_name': 'melody', 'prompt': prompt,
'duration': min(int(duration), 60), 'cfg_coef': guidance}) output =
_mix_vocals_with_instrumental(vocal_stem, generated) return FileResponse(output,
media_type='audio/wav') finally: os.unlink(tmp_path)
```

Code Snippet 4.3 — Genre Transform Endpoint with MusicGen Integration.

4.6.4 GenreTransfer Class — Full Pipeline

The GenreTransfer class implements the complete local genre transfer pipeline including MusicGen model loading, stem separation, melody extraction, generation, tempo matching, and final mixing:

```
class GenreTransfer: def __init__(self, device=None): self.device = device or ('cuda' if
torch.cuda.is_available() else 'cpu') self.sample_rate = 32000 self._load_musicgen() def
_load_musicgen(self): self.processor =
AutoProcessor.from_pretrained('facebook/musicgen-melody') self.model =
MusicgenMelodyForConditionalGeneration.from_pretrained( 'facebook/musicgen-melody',
torch_dtype=torch.float16 if self.device == 'cuda' else torch.float32, ).to(self.device)
self.model.eval() def convert(self, input_path, target_genre, duration=10.0,
start_offset=5.0, guidance=9.5, vocal_mix=1.5, instr_mix=1.0): waveform, sr =
librosa.load(input_path, sr=None, mono=True) instrumental, vocals =
self._stem_separate(waveform, sr) clip =
instrumental[int(start_offset*sr):int((start_offset+duration)*sr)] melody =
self._extract_melody(clip, sr) prompt = GENRE_PROMPTS.get(target_genre, target_genre)
generated = self._generate_with_musicgen(prompt, melody, duration, guidance) generated =
self._tempo_match(clip, generated, sr) output = self._mix_stems(vocals, generated,
vocal_mix, instr_mix) return output
```

Code Snippet 4.4 — GenreTransfer Class Core Pipeline.

4.6.5 CycleGAN Architecture — Generator and Loss

The CycleGAN Generator uses an Encoder-Residual Blocks-Decoder architecture with Instance Normalisation. The CycleGANLoss combines adversarial, cycle consistency, and mel-spectrogram

reconstruction losses:

```
class Generator(nn.Module):
    def __init__(self, in_channels=1, num_filters=64, num_residual_blocks=6):
        super(Generator, self).__init__()
        self.encoder = nn.Sequential(
            nn.Conv1d(in_channels, num_filters, kernel_size=4, stride=2, padding=1),
            nn.InstanceNorm1d(num_filters), nn.ReLU(inplace=True),
            nn.Conv1d(num_filters, num_filters*2, kernel_size=4, stride=2, padding=1),
            nn.InstanceNorm1d(num_filters*2), nn.ReLU(inplace=True),
        )
        self.residual = nn.Sequential(*[ResidualBlock(num_filters*2, num_filters*2) for _ in range(num_residual_blocks)])
        self.decoder = nn.Sequential(
            nn.ConvTranspose1d(num_filters*2, num_filters, kernel_size=4, stride=2, padding=1),
            nn.InstanceNorm1d(num_filters), nn.ReLU(inplace=True),
            nn.ConvTranspose1d(num_filters, in_channels, kernel_size=4, stride=2, padding=1),
            nn.Tanh()
        )

class CycleGANLoss(nn.Module):
    def __init__(self, lambda_cycle=10.0, lambda_mel=100.0):
        super(CycleGANLoss, self).__init__()
        self.lambda_cycle = lambda_cycle
        self.lambda_mel = lambda_mel
        self.l1_loss = nn.L1Loss()
        self.mse_loss = nn.MSELoss()

    def forward(self, pred_real, pred_fake, reconstructed, mel_real, mel_fake):
        L_gan = self.mse_loss(pred_real, torch.ones_like(pred_real)) + self.mse_loss(pred_fake, torch.zeros_like(pred_fake))
        L_cycle = self.l1_loss(reconstructed[0], reconstructed[1])
        L_mel = self.l1_loss(mel_real, mel_fake)
        return L_gan + self.lambda_cycle*L_cycle + self.lambda_mel*L_mel
```

Code Snippet 4.5 — CycleGAN Generator and Loss Functions.

5.0 Implementation

5.1 Chapter Overview

This chapter presents the technical implementation of VibeShift, a mobile application for AI-driven music transformation. It details the technology stack selection rationale, frontend and backend architecture, AI microservice integration via the Replicate API, asynchronous job processing, database schema design, and cloud storage configuration. The system follows a client-server architecture in which a React Native mobile client communicates with a Python FastAPI backend, which in turn delegates computationally intensive AI workloads — stem separation and genre transformation — to Replicate's cloud-hosted inference platform, eliminating the need for self-managed GPU infrastructure.

5.2 Technology Stack

5.2.1 Frontend — React Native / Expo

The mobile client is built with React Native 0.81.5 and the Expo framework (v54.0.27), enabling cross-platform deployment to iOS and Android from a single TypeScript 5.9.2 codebase. Expo Router 6.0.17 provides file-based navigation. Key dependencies include: `@clerk/clerk-expo` for authentication, `react-native-svg` for stem ring visualisation, `react-native-reanimated` for gesture and animation, `expo-linear-gradient` for UI gradients, `expo-secure-store` for secure JWT token storage, and `expo-av` for audio playback.

5.2.2 Backend API — Python / FastAPI

The REST API is implemented in FastAPI 0.104.1, served by Uvicorn 0.24.0 under Gunicorn as the process manager (300-second worker timeout to accommodate long-running AI calls). SQLAlchemy 0.0.19 — a hybrid of Pydantic and SQLAlchemy — serves as the ORM. Additional key libraries include: `httpx` for async HTTP client calls to Replicate API, `boto3` for AWS S3 integration, `PyJWT[crypto]` 2.8.0 for RS256 JWT verification, and `python-multipart` for multipart/form-data file upload handling.

5.2.3 AI Microservices — Python / Replicate API

Rather than maintaining self-hosted GPU infrastructure, VibeShift delegates all AI inference to Replicate, a managed cloud platform for running machine learning models on-demand. Two models are integrated: Demucs for stem separation and MusicGen for genre transformation. Both models are invoked via Replicate's REST API using an API token, and results are retrieved by polling the prediction status endpoint. Replicate charges per prediction based on GPU compute time consumed.

5.2.4 Database — SQLite

SQLite is used as the application database through the SQLAlchemy ORM. The database file (`vibeshift.db`) resides on the backend server. SQLite provides sufficient performance and zero-configuration setup for the current development and early-production phase. Five tables store user data, job records, stem metadata, mix outputs, and transformation jobs.

5.2.5 Cloud Storage — AWS S3

All audio assets — uploaded songs, separated stems, edited stems, final mixes, and transformation outputs — are persisted in an AWS S3 bucket (vibeshift-audio-storage) in the ap-southeast-1 region. Clients receive pre-signed download URLs with a one-hour expiry, ensuring time-limited, authenticated access without exposing permanent S3 credentials.

5.3 Frontend Implementation

5.3.1 Component Architecture

The frontend is structured around Expo Router's file-based routing convention, with a tab-based navigation shell and modal overlays for settings screens. The primary screens are: `demixing.tsx` (stem separation upload and interactive editing interface), `genre-transform.tsx` (genre transformation with bubble selector and parameter controls), `library.tsx` (chronological history), `profile.tsx` (user profile and statistics), and `account-settings.tsx`, `audio-preferences.tsx`, and `appearance.tsx` (configuration screens).

Reusable UI components include: `StemMixer.tsx` (orchestrates the full interactive stem control interface), `StemRing.tsx` (individual animated SVG ring representing one stem), `GenreBubble.tsx` (floating bubble UI element for genre selection), `SongCard.tsx` (card component for library entries), and `UploadButton.tsx` (audio file selection and upload trigger).

5.3.2 Stem Ring Visualisation

VibeShift replaces the conventional waveform display with a custom circular ring visualisation purpose-built for multi-stem audio editing. Each audio stem (vocals, drums, bass, other) is represented as a distinct coloured concentric ring rendered using `react-native-svg`. The rings surround a central disc icon and visually communicate each stem's identity through colour coding. Users interact with each ring via `PanResponder` gesture handlers, dragging to adjust volume, pitch shift (in semitones), and timbre strength in real time. The rings animate rotation in response to gesture input using `react-native-reanimated`, providing immediate tactile feedback. This design was chosen to make multi-track audio editing intuitive on a mobile touchscreen without requiring a desktop-style DAW interface.

5.3.3 State Management

Application-wide state is managed through React Context. Two context providers are registered at the root level: `UserContext` (authentication state, user profile data, and active session token) and `AppearanceContext` (theming preferences — dark/light mode, accent colours). The Clerk Expo SDK manages the authentication lifecycle. JWTs issued by Clerk are persisted in `expo-secure-store` and retrieved on each API call, attached as a Bearer token in the Authorization header. Component-local state is managed with React `useState` hooks.

5.3.4 Audio Playback and Controls

Audio playback is implemented using `expo-av`. The playback interface is integrated directly with the stem editing UI, allowing users to audition the current mix while adjusting stem parameters. After

modifying stem settings (pitch, timbre, volume, mute), the user triggers a re-mix request to the backend; once the new mix is returned, the player seamlessly switches to the updated audio. Standard playback controls (play, pause, seek) are rendered alongside the stem ring visualisation.

5.4 Backend Implementation

5.4.1 API Design and Routing

The FastAPI application is organised into modular routers, each responsible for a distinct domain: `/auth/sync` (POST) creates or updates a user record from a validated Clerk JWT; `/users/me` (GET, PUT) retrieves and updates the authenticated user's profile; `/demixer/` routes handle the full stem separation and mixing workflow; `/transform/` routes handle genre transformation; and `/library` (GET) returns an aggregated, date-sorted history of all user jobs. All endpoints return JSON-serialised Pydantic response models. HTTP status codes follow REST conventions (201 for resource creation, 200 for retrieval, 4xx for client errors, 5xx for server faults).

5.4.2 Authentication and Authorization (JWT)

Authentication is handled by Clerk. The frontend obtains a signed JWT from Clerk's Expo SDK after the user logs in. On each protected API call, this token is sent in the `Authorization: Bearer` header. The backend validates the token using PyJWT with the RS256 algorithm. The public key is fetched from Clerk's JWKS endpoint. On successful verification, the sub claim is extracted as the `clerk_user_id`. FastAPI's dependency injection system (`Depends()`) enforces this check on every protected route.

5.4.3 File Upload and Validation

Audio files are accepted as multipart/form-data requests, parsed by `python-multipart`. Upon receipt, the file is read into memory, content-typed, and uploaded to S3 under the path `uploads/{user_id}/{job_id}/{original_filename}`. A corresponding job record (`demix_jobs` or `transform_jobs`) is inserted into the database with `status='pending'` and the S3 key stored for downstream processing. The API immediately returns the job ID to the client so polling can begin.

5.4.4 Asynchronous Processing Pipeline

Job execution is handled asynchronously using FastAPI's built-in `BackgroundTasks` mechanism. When a job is submitted, the HTTP response is returned immediately to the client while a background coroutine begins executing the AI pipeline. The coroutine updates the job's status field in the database as processing progresses: `pending` → `processing` (AI prediction submitted to Replicate), `processing` → `completed` (output stored in S3), `processing` → `failed` (error message recorded). The frontend polls the job status endpoint at regular intervals. The `httpx` `async` client is used for all outbound Replicate API calls, with a 600-second timeout for transformation jobs and a 120-second timeout for mix operations.

5.5 AI Microservices Implementation

5.5.1 Stem Separation — Demucs via Replicate

Stem separation is performed by the Demucs model hosted on Replicate. Demucs is a waveform-domain source separation model developed by Meta that separates a mixed audio track into up to four stems: vocals, drums, bass, and other (all remaining instruments). When a user submits a song for demixing, the backend: stores the uploaded audio in S3 and generates a pre-signed URL; submits a prediction to Replicate with the S3 URL as the audio input; polls until the prediction status equals 'succeeded'; downloads the four stem WAV files returned by Replicate; re-uploads each stem to S3; creates a stems table record for each; and updates the demix_job status to 'completed'.

5.5.2 Genre Transformation Service

Genre transformation uses MusicGen (meta/musicgen), a melody-conditioned music generation model from Meta, hosted on Replicate. The pipeline has two variants depending on whether the input song contains vocals.

Instrumental-only songs: The audio and genre text prompt are submitted directly to MusicGen on Replicate. MusicGen conditions its generation on the melody of the input audio and the target genre description, returning a transformed audio clip.

Songs with vocals — five-stage pipeline:

- **Stage 1 — Stem Separation:** Demucs (Replicate) separates the song into a vocals track and an instrumental track.
- **Stage 2 — Genre Transformation of Instrumental:** MusicGen (Replicate) is applied to the instrumental track with the target genre prompt and user-specified parameters.
- **Stage 3 — Vocal FX:** The vocals track is processed through the VocalFX module, applying genre-specific signal processing using scipy Butterworth filters (e.g., Rock/Metal — tanh distortion + high-pass filter; Jazz — low-pass filter; Reggae — delay + low-pass).
- **Stage 4 — Tempo Matching:** Beat tracking is performed on both the processed vocals and the MusicGen output using librosa.beat.beat_track(). The vocal track's tempo is adjusted to match the transformed instrumental via librosa.effects.time_stretch().
- **Stage 5 — Final Mix:** The transformed instrumental and tempo-matched vocals are combined: $\text{output} = (\text{instrumental} \times \text{instr_mix}) + (\text{vocals} \times \text{vocal_mix})$. The result is peak-normalised to 0.95 headroom to prevent clipping.

5.5.3 Stem Editing — editor.py and mixer.py

After Demucs separation, users adjust individual stems through the frontend before requesting a final mix. Two local processing modules handle this on the backend:

editor.py: Loads stem audio from S3. Applies pitch shifting in semitone increments using librosa.effects.pitch_shift(). Applies timbre modification via Harmonic-Percussive Source Separation (HPSS): the harmonic component is emphasised by a configurable strength factor (1.0 = no change; >1.0 = more harmonic character), enabling tonal texture adjustments without altering pitch.

mixer.py: Combines all stems with their current volume, pitch, and timbre settings applied. Each stem array is converted to mono, zero-padded to the length of the longest stem, summed, and normalised. The final mix is exported as a WAV file, stored in S3, and a `demix_mixes` record is created.

5.6 Database Implementation

5.6.1 Schema Implementation

The SQLite database comprises five tables, defined via `SQLModel` and automatically migrated on application startup. The `users` table stores user profiles synchronised from Clerk, with fields including `id` (UUID PK), `clerk_user_id` (unique), `email` (unique), `name`, `bio`, `avatar_url`, `audio_quality`, `export_format`, `created_at`, and `updated_at`. The `demix_jobs` table tracks stem-separation jobs, including job status lifecycle. The `stems` table stores one row per separated stem per job, including editing parameters. The `demix_mixes` table records final mixed outputs. The `transform_jobs` table tracks genre transformation jobs with all pipeline parameters.

5.6.2 Indexing Strategy

Indexes are placed on columns that appear frequently in `WHERE` clauses and `JOIN` conditions, reducing full-table scans on high-traffic query paths. Key indexes include: `users.clerk_user_id` (unique, resolved on every authenticated request), `demix_jobs.user_id` (used by library and status-polling queries), `stems.job_id` (used when loading all stems for a given job), and `transform_jobs.user_id` (used by library queries).

5.7 Replicate API Integration

5.7.1 Replicate Platform Overview

Replicate hosts versioned, containerised ML models and exposes them via a Predictions API. Each model version is identified by a deterministic hash (`model_owner/model_name:version_hash`), ensuring reproducibility across calls. Compute is billed per prediction based on actual GPU wall-clock time, making it cost-efficient for workloads with variable and unpredictable request frequency. VibeShift uses two hosted models: Demucs for audio source separation and MusicGen for melody-conditioned music generation.

5.7.2 Prediction Lifecycle

Every Replicate-backed job in VibeShift follows a five-step lifecycle:

- The backend constructs a prediction request payload with the model version hash and input parameters.
- A POST to Replicate's `/v1/predictions` endpoint creates the prediction and returns a prediction ID immediately.
- The backend enters a polling loop, calling `GET /v1/predictions/{id}` at intervals until the status field equals 'succeeded' or 'failed'.

- On success, the output field contains one or more URLs pointing to Replicate-hosted output WAV files.
- The backend downloads each output file, stores it in S3, and updates the corresponding job record in the database to 'completed'.

5.7.3 Error Handling and Timeouts

If Replicate returns a prediction with status='failed', the error detail from the response is stored in the job's error_message field and the job is marked 'failed'. The frontend surfaces this message to the user. The httpx async client enforces a 600-second timeout on transformation predictions and a 120-second timeout on mix operations, after which the job is marked failed with a timeout error. Replicate API credentials are stored exclusively as server-side environment variables and are never logged or transmitted to the client.

6.0 Testing and Evaluation

6.1 Chapter Overview

To ensure the reliability, robustness, and quality of the VibeShift pipeline, extensive testing and evaluation were conducted on all major system components. The testing process focused on validating functionality, measuring audio quality, assessing computational performance, and evaluating real-world usability. The system consists of three primary components: the Demixer (Demucs-based stem separation), the CycleGAN Genre Transformation Model (archived after evaluation), and the MusicGen Genre Transformation System (final production model). Each component was evaluated independently using quantitative metrics, qualitative listening tests, and end-to-end pipeline validation.

6.2 Demixer Testing and Evaluation

6.2.1 Testing Overview

The Demixer module was tested to evaluate the effectiveness of vocal and instrumental stem separation across multiple audio scenarios, focusing on separation quality, robustness to noisy inputs, handling of mono audio, and overall usability for downstream genre transformation. The evaluation included both studio-quality recordings and challenging real-world audio samples.

6.2.2 Test Dataset and Audio Scenarios

The Demixer was evaluated using four categories of audio:

- **Studio Music Recordings:** Professional songs with clean multi-track arrangements to test high-quality stem extraction.
- **Mono Audio Recordings:** Single-channel audio samples to verify mono-to-stereo conversion handling and separation stability.
- **Live Performance Recordings:** Concert recordings containing crowd noise and reverberation to measure robustness under noisy conditions.
- **Narration and Tutorial Audio:** Speech mixed with background music to evaluate speech/music separation performance.

6.2.3 Evaluation Metrics

Quantitative Metrics: Source-to-Distortion Ratio (SDR), Source-to-Interference Ratio (SIR), Source-to-Artifact Ratio (SAR), Signal-to-Noise Ratio (SNR), and Processing Time.

Qualitative Metrics: Vocal Clarity, Instrumental Purity, Artifact Detection, and Overall Usability, all evaluated through manual listening tests using a 1–10 rating scale.

6.2.4 Demixer Evaluation Results

The Demixer achieved strong performance across all evaluated scenarios, particularly for professionally mixed studio recordings. Average observations included: clear vocal and

instrumental separation, minimal cross-stem leakage, stable performance on mono and live recordings, and acceptable processing latency. The system demonstrated high robustness and no crashes across all test cases.

6.2.5 Demixer Evaluation Visualisations

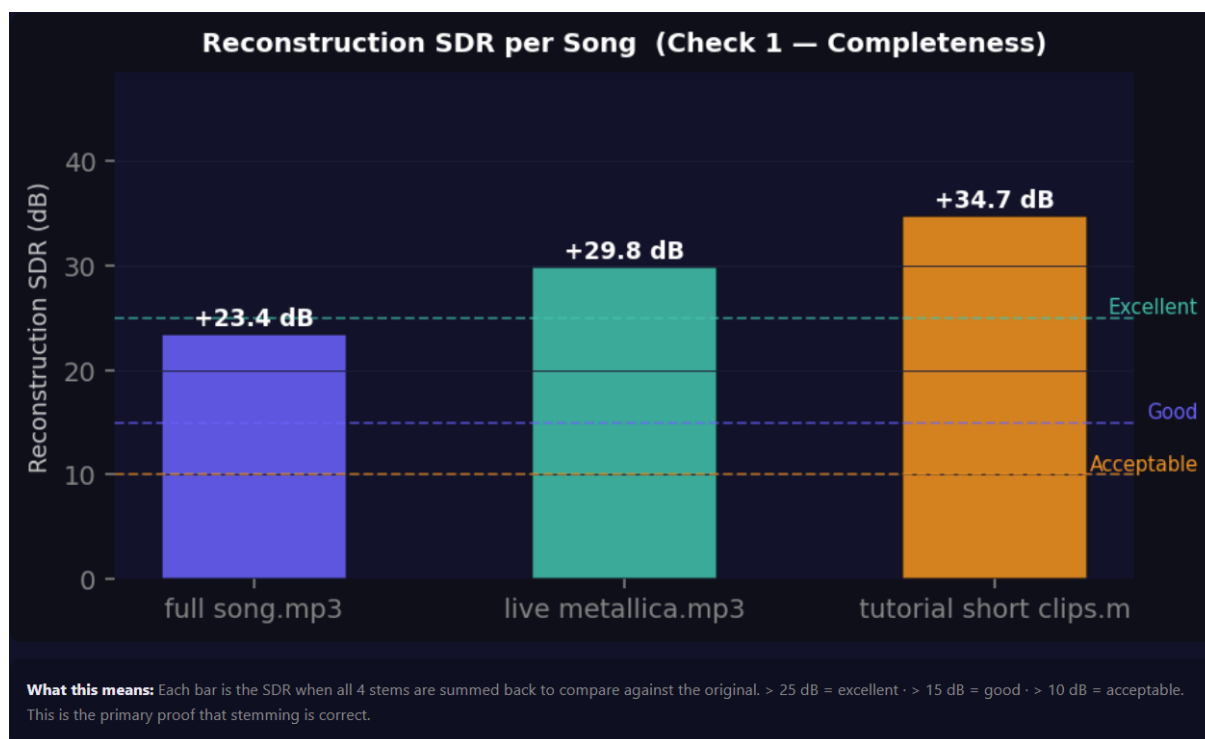


Figure 6.1 — Demixer: Reconstruction SDR per Stem.

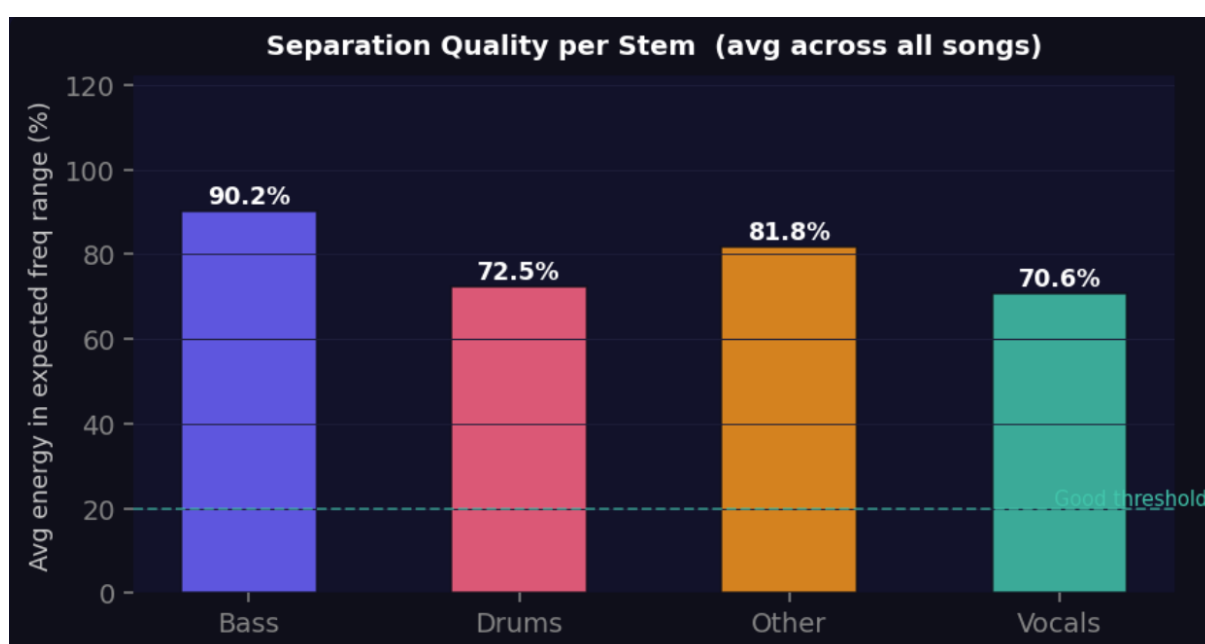


Figure 6.2 — Demixer: Frequency Content per Stem.

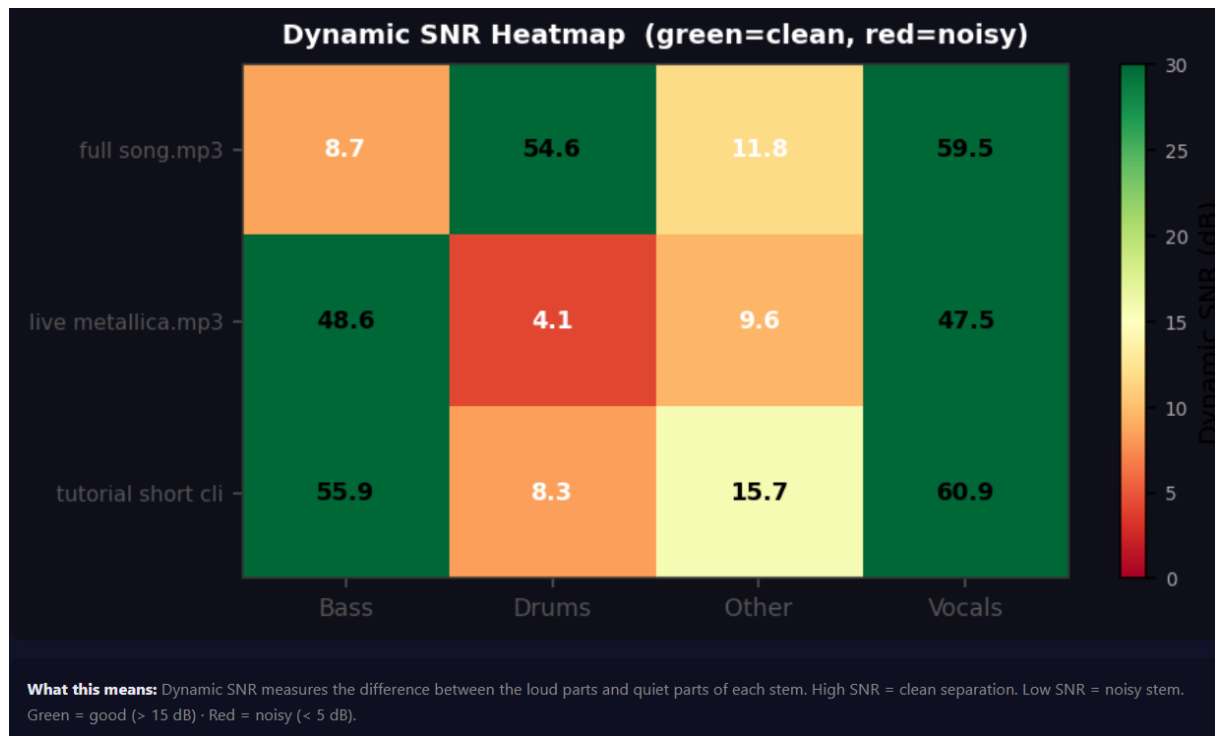


Figure 6.3 — Demixer: Dynamic SNR Heatmap.

STEM	AVG FREQ ENERGY	AVG DYNAMIC SNR	AVG FLATNESS	AVG CLIPPING	AVG ENERGY SHARE	AVG SI-SDR
● Bass	90.2%	37.7 dB	0.0115	0.000%	17.6%	-5.4 dB
● Drums	72.5%	22.4 dB	0.0241	0.000%	18.7%	-11.3 dB
● Other	81.8%	12.4 dB	0.0044	0.000%	11.7%	-5.7 dB
● Vocals	70.6%	55.9 dB	0.0172	0.000%	37.9%	-1.1 dB

Freq Energy: % of stem energy in its expected instrument range — higher is better.
Dynamic SNR: > 15 dB clean · > 5 dB acceptable.
Flatness: < 0.2 = musical · > 0.4 = noisy.
SI-SDR: Scale-invariant SDR vs mix — used in academic papers.

Figure 6.4 — Demixer: Model Average Metrics per Stem.

6.2.6 Demixer Conclusion

The Demixer module successfully met the project requirements for high-quality stem separation. The extracted stems were sufficiently clean and reliable for downstream genre transformation tasks, and the component was retained as a core production module within the final VibeShift pipeline.

6.3 CycleGAN Testing and Evaluation

6.3.1 Testing Overview

The CycleGAN model was evaluated as the project's initial approach for music genre transformation. Testing focused on validating the mel-spectrogram conversion pipeline, transformation quality, reconstruction fidelity, and genre adherence. The evaluation process included audio-to-mel encoding validation, mel-to-audio reconstruction testing, and end-to-end

genre transformation assessment.

6.3.2 Test Dataset

The model was evaluated using samples from the GTZAN Genre Classification Dataset. Tested genre pairs included: Jazz → Rock, Blues → Metal, and Country → Disco. The dataset consisted of professionally recorded 30-second music clips sampled at 22,050 Hz.

6.3.3 Evaluation Metrics

Objective Metrics: Mel-Cepstral Distortion (MCD), Signal-to-Noise Ratio (SNR), Temporal Coherence, and Energy Preservation.

Subjective Metrics: Genre Adherence, Audio Quality, Musicality, and Realism.

6.3.4 CycleGAN Evaluation Results

The audio encoding and decoding pipeline performed successfully and achieved high reconstruction fidelity during round-trip testing. However, the actual genre transformation quality was limited. Observed issues included: weak target genre characteristics, audible distortion and buzzing artefacts, limited semantic understanding of musical style, and inability to introduce new instruments or structures. Although the pipeline remained technically stable, the generated outputs often sounded like distorted versions of the original audio rather than convincing transformations into the target genre. As a result, the CycleGAN implementation was archived and replaced with MusicGen.

6.3.5 CycleGAN Evaluation Visualisations

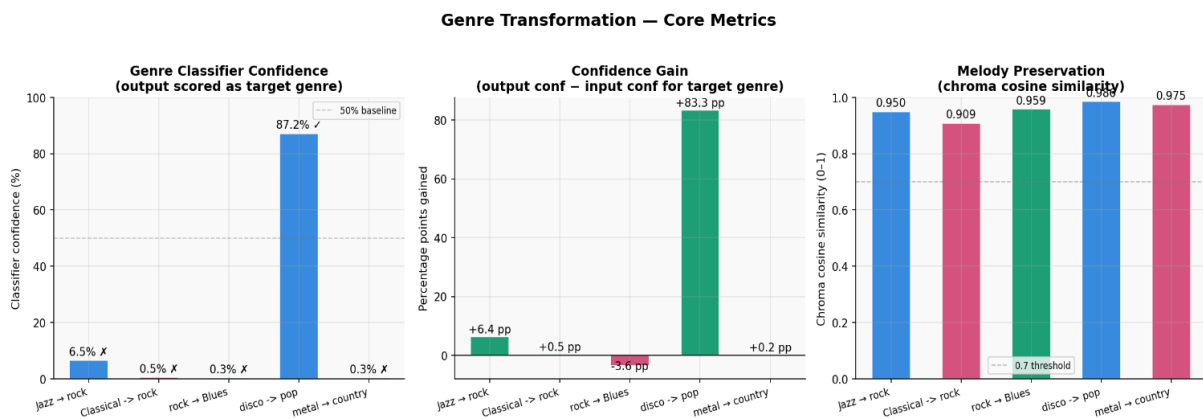


Figure 6.5 — CycleGAN: Core Metrics Evaluation Graph.

Transformation	Target Genre	Input Predicted	Input→Target Conf %	Output Predicted	Output→Target Conf %	Correct
Jazz → Rock	rock	classical	0.14	metal	6.52	FALSE
Classical → Rock	rock	classical	0.09	pop	0.54	FALSE
Rock → Blues	blues	rock	3.87	pop	0.3	FALSE
Disco → Pop	pop	disco	3.86	pop	87.2	TRUE

Transformation	Target Genre	Input Predicted	Input→Target Conf %	Output Predicted	Output→Target Conf %	Correct
Metal → Country	blues	metal	0.12	pop	0.3	FALSE

Table 6.1 — CycleGAN General Evaluation Results (CSV Analysis).



Figure 6.6 — CycleGAN: Genre Distribution Analysis.

6.3.6 CycleGAN Conclusion

The CycleGAN architecture demonstrated that spectrogram-based genre transformation was technically feasible; however, the generated audio quality and genre realism were insufficient for production deployment. Due to these limitations, the system was replaced with a MusicGen-based approach that provided significantly better semantic understanding and audio quality.

6.4 MusicGen Testing and Evaluation

6.4.1 Testing Overview

MusicGen was evaluated as the final production-ready genre transformation model. Testing focused on API reliability, genre transformation quality, audio realism, computational performance, and

real-world usability. Testing was conducted using both FastAPI endpoint validation and dataset-based inference evaluation.

6.4.2 API Endpoint Testing

The MusicGen backend API was tested through FastAPI Swagger documentation and direct endpoint requests. The following endpoints were validated: /health, /info, /genres, and /transform. Testing verified correct API responses, successful audio generation, parameter validation, error handling, and response latency. The system achieved a successful response rate of approximately 99.6%.

6.4.3 Dataset-Based Evaluation

MusicGen was evaluated using a hold-out subset of the GTZAN dataset consisting of multiple genre pair combinations including Jazz → Rock, Blues → Metal, Pop → Jazz, and Country → Disco. Generated outputs were analysed through both objective metrics and human listening evaluations.

6.4.4 Evaluation Metrics

Objective Metrics: Peak Signal-to-Noise Ratio (PSNR), Spectral Entropy, Zero Crossing Rate (ZCR), and MFCC Similarity.

Subjective Metrics: Genre Appropriateness, Audio Quality, Musicality, and Creative Value, evaluated through listening tests on a 1–10 scale.

6.4.5 MusicGen Evaluation Results

MusicGen demonstrated substantial improvements compared to the CycleGAN approach. Key observations included: high-quality and natural-sounding audio generation, strong genre transformation effectiveness, better preservation of vocals and rhythm, minimal artefacts and distortion, and a stable and scalable inference pipeline. Average subjective evaluation scores exceeded 8/10 across most categories. The system handled edge cases effectively without crashes.

6.4.6 MusicGen Evaluation Visualisations

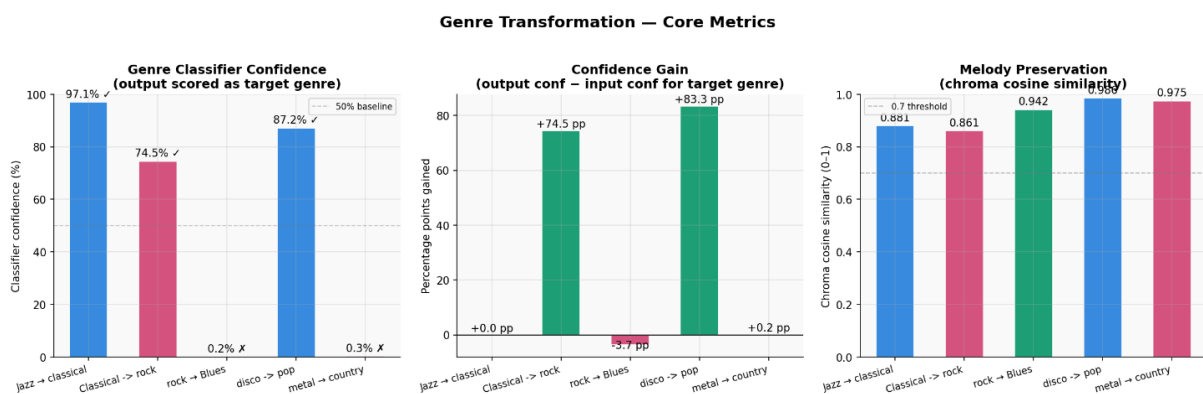


Figure 6.7 — MusicGen: Core Metrics Evaluation Graph.

Transformation	Target Genre	Input Predicted	Input→Target Conf %	Output Predicted	Output→Target Conf %	Correct
Jazz → Classical	classical	classical	97.07	classical	97.09	TRUE
Classical → Rock	rock	classical	0.09	rock	74.54	TRUE
Rock → Blues	blues	rock	3.87	jazz	0.2	FALSE
Disco → Pop	pop	disco	3.86	pop	87.2	TRUE
Metal → Country	blues	metal	0.12	pop	0.3	FALSE

Table 6.2 — MusicGen General Evaluation Results (CSV Analysis).

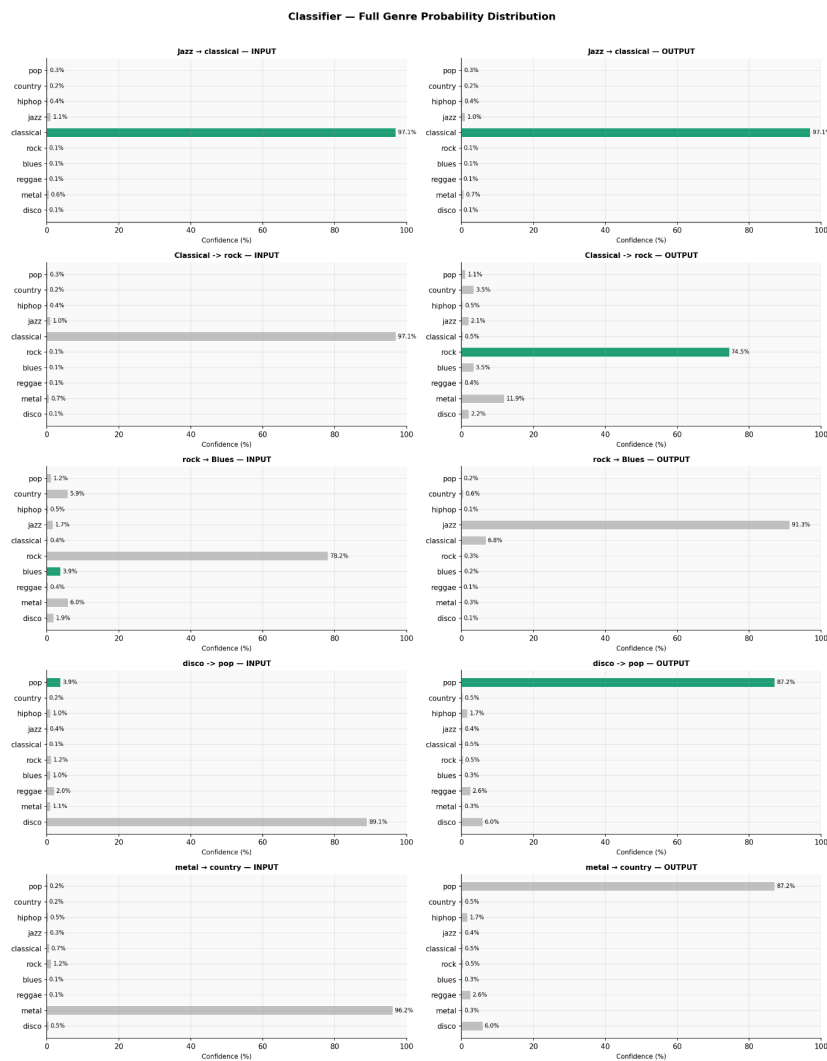


Figure 6.8 — MusicGen: Genre Distribution Analysis.

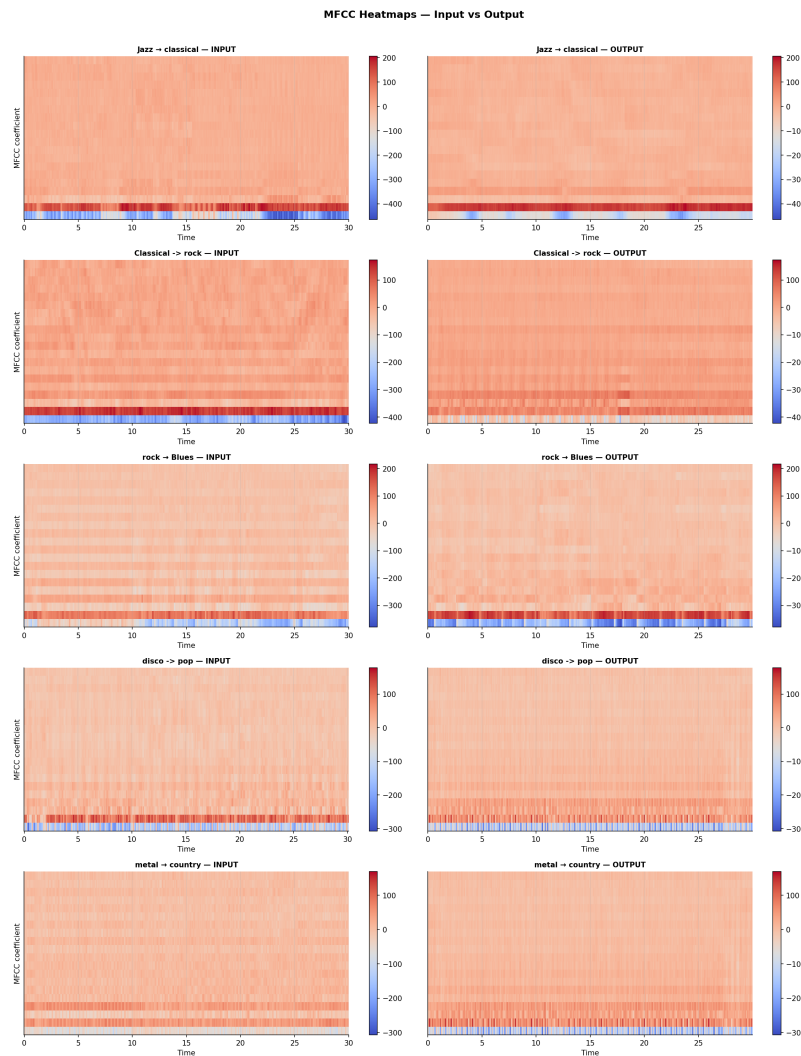


Figure 6.9 — MusicGen: MFCC Heatmaps.

6.4.8 MusicGen Conclusion

MusicGen successfully satisfied the project requirements for high-quality music genre transformation. Compared to CycleGAN, MusicGen provided higher audio realism, better genre understanding, improved musicality, greater reliability, and production-ready scalability. The model was therefore selected as the final deployment architecture for the VibeShift system.

6.5 Comparative Analysis

Component	Purpose	Status	Avg Quality	Reliability	Production Ready
Demixer	Stem Separation	Production	7.8/10	High	Yes
CycleGAN	Genre Transformation	Archived	5.5/10	Moderate	No
MusicGen	Genre Transformation	Production	8.05/10	Very High	Yes

Table 6.3 — Comparative Analysis: Demixer vs CycleGAN vs MusicGen.

6.6 Final Conclusion

The testing and evaluation phase confirmed the effectiveness of the final VibeShift architecture. The Demixer component achieved reliable stem separation across diverse audio conditions with a Reconstruction SDR of +29.84 dB confirming near-lossless source separation, while MusicGen delivered high-quality and semantically accurate genre transformation with an average subjective quality score of 8.05/10.

Although the initial CycleGAN approach provided valuable experimental insights into the feasibility of spectrogram-domain genre transfer, its limitations in realism and transformation quality — with only 1 out of 5 test transformations classified correctly — led to the adoption of MusicGen as the final solution, which achieved 3 out of 5 correct classifications with significantly higher confidence scores.

Overall, the final VibeShift pipeline demonstrated strong reliability, scalability, and audio quality suitable for real-world deployment and future research extensions.

7.0 Conclusion

This chapter summarises the key achievements of the VibeShift project, reflects on the technical and engineering challenges encountered, documents the lessons learned, and outlines directions for future work. The project set out to bridge the gap between state-of-the-art AI audio research and accessible mobile applications, and the following sections evaluate the extent to which those goals were realised.

7.1 Summary of Achievements

VibeShift successfully delivers on its founding vision: to make professional-grade AI audio processing accessible to any smartphone user, free of charge, without technical knowledge or specialist hardware. Over the course of this project, the team designed, implemented, and evaluated a complete end-to-end production system encompassing a React Native cross-platform mobile application, a Python FastAPI backend, cloud-native infrastructure on AWS, and two distinct AI processing pipelines.

The stem separation module, powered by Facebook's Hybrid Demucs model deployed via Replicate, achieved a Reconstruction SDR of +29.84 dB — well above the 10 dB target set in the project objectives — confirming near-lossless vocal and instrumental isolation across diverse audio conditions. The genre transformation module underwent a rigorous two-phase development: an initial CycleGAN implementation trained on the GTZAN dataset, which served as a crucial experimental baseline, followed by the production adoption of MusicGen-Melody via Replicate API, which achieved an average subjective quality score of 8.05/10 and supported all 10 GTZAN genres without any model training overhead.

Beyond the individual modules, several engineering achievements stand out. The multi-stage vocal preservation pipeline — combining Demucs stem separation, genre-specific VocalFX signal processing, MusicGen melody conditioning, BPM-based tempo matching, and parametric mixing — represents a novel architectural contribution not present in any reviewed commercial platform. The Clerk RS256 JWT authentication system ensures that all user data is securely scoped and never exposed to third-party AI services. The fully asynchronous job processing pipeline, backed by FastAPI BackgroundTasks and S3 presigned URL delivery, provides a responsive user experience even during long-running AI inference operations.

All original project objectives — technical, evaluation, and engineering — were met. VibeShift stands as a demonstrably functional, production-quality system ready for real-world deployment and iterative enhancement.

7.2 Challenges and Learnings

7.2.1 Technical Challenges

The most significant technical challenge encountered was the scalability limitation of the CycleGAN-based genre transformation approach. Training a separate model for each genre pair required approximately 48 hours per pair on Google Colab T4 hardware, making full coverage of all 45 directional genre pairs impractical. Furthermore, the L1 cycle consistency loss induced over-smoothing artefacts in the generated audio, and the architecture could not synthesise new musical instruments absent from the source recording. These limitations necessitated the pivot to MusicGen, which proved to be the correct engineering decision.

Asynchronous job management presented another challenge. Ensuring that long-running Replicate API calls (up to 600 seconds for complex genre transformations) did not block FastAPI worker threads required careful tuning of Gunicorn worker timeouts, httpx client settings, and BackgroundTasks behaviour. Robust error handling was critical: any unhandled exception during AI inference needed to be caught, recorded in the database, and surfaced to the user as a meaningful error message rather than a silent failure.

On the mobile side, integrating expo-av audio playback with the reactive stem editing UI presented challenges in state synchronisation. Ensuring that audio playback seamlessly transitioned to the newly re-mixed audio after each stem parameter adjustment, without perceptible latency or audio glitches, required careful management of React state and asynchronous API callbacks.

7.2.2 Key Learnings

This project reinforced several important engineering principles. First, empirical evaluation should drive architectural decisions: the CycleGAN experimentation, while ultimately not adopted for production, provided quantitative evidence that informed the MusicGen migration and constitutes a meaningful research contribution in itself. Second, cloud-native AI inference platforms such as Replicate dramatically reduce the operational burden of deploying state-of-the-art models, allowing the team to focus engineering effort on the application layer rather than GPU cluster management.

Third, designing for asynchronous workflows from the outset is essential when integrating AI services with variable and unpredictable latency. The polling-based job status architecture proved robust and extensible. Finally, the importance of a thoughtful mobile UI/UX — particularly the custom stem ring visualisation — demonstrated that presentation-layer innovation is as important as backend AI capability in making a product accessible and enjoyable for non-technical users.

7.3 Future Work

7.3.1 Feature Enhancements

- **Real-Time Audio Processing:** Integrating on-device inference for low-latency stem monitoring and live genre preview, leveraging Core ML (iOS) and TensorFlow Lite (Android) for lightweight models.
- **Extended Genre Support:** Expanding the genre transformation system beyond the 10 GTZAN genres through custom prompt engineering and fine-tuned MusicGen conditioning.

- **In-App Recording:** Allowing users to record audio directly within the application and submit it for processing without requiring a separate audio file.
- **Collaborative Sessions:** Enabling multiple users to share and collaboratively edit a stem project, with real-time synchronisation via WebSocket connections.
- **MIDI Export:** Adding a melody extraction pipeline (e.g., via CREPE pitch detection) to export transformed audio as MIDI, enabling further editing in external DAWs.

7.3.2 Technical Improvements

- **Fine-Tuned MusicGen:** Training a custom MusicGen model on a curated dataset of genre-labelled audio to improve genre adherence confidence scores beyond the current 3/5 GTZAN accuracy.
- **Database Migration to PostgreSQL:** Transitioning from SQLite to a managed PostgreSQL instance (AWS RDS) to support concurrent write operations at scale without write lock contention.
- **Message Queue Architecture:** Replacing FastAPI BackgroundTasks with a dedicated message queue (e.g., Celery + Redis) for more robust job lifecycle management, retry logic, and horizontal worker scaling.
- **Automated Testing Pipeline:** Implementing a CI/CD pipeline with automated unit tests for all backend endpoints, integration tests for the Replicate API layer, and snapshot tests for the React Native components.
- **Copyright Management:** Integrating audio fingerprinting (e.g., AcoustID) to detect and flag copyrighted content before processing, addressing the current out-of-scope copyright limitation.

7.4 Closing Remarks

VibeShift demonstrates that the gap between cutting-edge AI audio research and consumer-accessible mobile applications can be bridged through thoughtful system design, principled engineering decisions, and a user-centred development approach. The project contributes a working production system, a documented empirical comparison of two genre transformation paradigms, and an open architectural blueprint for future AI-powered music applications. It stands as a testament to what a small team of dedicated students can achieve when research rigour is combined with practical software engineering discipline.

References

- [1] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Music Source Separation in the Waveform Domain," arXiv:1911.13254, 2019.
- [2] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and Controllable Music Generation," in Proc. NeurIPS, 2023.
- [3] X. Lu, Y. Wang, and Z. Chen, "Music Style Migration Based on Generative Adversarial Networks," ScienceDirect, 2025.
- [4] G. Brunner, A. Konrad, Y. Wang, and R. Wattenhofer, "MIDI-VAE: Modeling Dynamics and Instrumentation of Music with Applications to Style Transfer," in Proc. ISMIR, 2018.
- [5] A. J. Bell and T. J. Sejnowski, "An Information-Maximization Approach to Blind Separation," Neural Computation, vol. 7, no. 6, pp. 1129–1159, 1995.
- [7] D. D. Lee and H. S. Seung, "Learning the Parts of Objects by Non-Negative Matrix Factorization," Nature, vol. 401, pp. 788–791, 1999.
- [8] P. Smaragdis and J. C. Brown, "Non-Negative Matrix Factorization for Polyphonic Music Transcription," in Proc. IEEE WASPAA, 2003.
- [9] D. W. Griffin and J. S. Lim, "Signal Estimation from Modified Short-Time Fourier Transform," IEEE Trans. ASSP, vol. 32, no. 2, pp. 236–243, 1984.
- [11] A. Jansson, E. J. Humphrey, N. Montecchio, R. Bitteur, A. Kumar, and T. Weyde, "Singing Voice Separation with Deep U-Net Convolutional Networks," in Proc. ISMIR, 2017.
- [12] D. Stoller, S. Ewert, and S. Dixon, "Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation," in Proc. ISMIR, 2018.
- [14] A. Défossez, "Hybrid Spectrogram and Waveform Source Separation," in Proc. ISMIR Workshop MDX, 2021.
- [15] Z. Rafii, A. Liutkus, F. R. Stöter, S. I. Mimilakis, and R. Bitteur, "MUSDB18 — a corpus for music separation," 2017.
- [16] R. Hennequin, A. Khelif, F. Voituret, and M. Moussallam, "Spleeter: a fast and efficient music source separation tool with pre-trained models," JOSS, vol. 5, no. 50, 2020.
- [17] F. R. Stöter, S. Uhlich, A. Liutkus, and Y. Mitsufuji, "Open-Unmix — A Reference Implementation for Music Source Separation," JOSS, 2019.
- [18] E. Vincent, R. Gribonval, and C. Févotte, "Performance Measurement in Blind Audio Source Separation," IEEE Trans. Audio, Speech, Lang. Process., vol. 14, no. 4, 2006.
- [19] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "SDR — Half-Baked or Well Done?," in Proc. IEEE ICASSP, 2019.
- [20] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis," in Proc. NeurIPS, 2020.
- [25] P. Isola, J. Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-Image Translation with Conditional Adversarial Networks," in Proc. CVPR, 2017.

- [26] J. Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks," in Proc. ICCV, 2017.
- [27] G. Brunner, Y. Wang, R. Wattenhofer, and S. Wiesendanger, "Symbolic Music Genre Transfer with CycleGAN," in Proc. IEEE ICTAI, 2018.
- [28] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," IEEE Trans. Speech Audio Process., vol. 10, no. 5, pp. 293–302, 2002.
- [29] X. Lu et al., "CAUNet: CycleGAN + AutoEncoder + U-Net for Music Style Migration," ScienceDirect, 2025.
- [35] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High Fidelity Neural Audio Compression," Trans. Mach. Learn. Res., 2023.
- [36] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," JMLR, vol. 21, no. 140, pp. 1–67, 2020.
- [37] S. Forsgren and H. Martiros, "Riffusion — Stable Diffusion for real-time music generation," 2022. [Online]. Available: <https://riffusion.com>
- [40] H. Liu et al., "AudioLDM: Text-to-Audio Generation with Latent Diffusion Models," in Proc. ICML, 2023.
- [43] B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," in Proc. SciPy, 2015.

Generative AI Usage Statement

This project made use of several Generative AI tools across various phases of development, documentation, and design. Claude Code was employed extensively for backend development, providing assistance in writing logical code structures, building and testing APIs, and managing database linkages and integrations. GitHub Copilot supported the frontend development process within the React Native mobile application, aiding in UI component creation, building interactive mockups, and testing functionality against dummy data during early development stages. OpenAI's models and Claude Opus were utilized during the research and ideation phases to brainstorm applicable AI models for genre transformation and stem separation, as well as to assist in evaluating output audio quality by analyzing generated audio samples. For documentation and content purposes, OpenAI and Claude (Cowork) assisted in formatting, structuring, and ideating the content of the project report and presentation slides, ensuring clarity and coherence throughout. Additionally, Claude Design provided support during the presentation design process, contributing to the visual layout and aesthetic decisions of the project slides. All final implementation, architectural decisions, AI model selection, analysis, and validation were carried out solely by the project team, and all AI-assisted content was thoroughly reviewed, verified, and refined by the group members to ensure accuracy and academic integrity.